Dhan HQ API Comprehensive Documentation

This document provides a comprehensive overview of the Dhan HQ API, including all available endpoints, their functionalities, request and response structures, and Python usage examples. The information is compiled from the official Dhan HQ API documentation and their official Python client GitHub repository.

API Overview

Dhan HQ API allows users to automate investing and trading with real-time order execution, position management, live and historical data, and tradebook access. It also provides real-time market data via DhanHQ Live Market Feed.

Authentication

All API requests require an `access-token` (JWT) and `client-id` in the request headers for authentication.

Python Client (`dhanhq-py`)

The official Python client simplifies interaction with the Dhan HQ API. It can be installed via pip:

```
pip install dhanhq
```

Initialization

Starting from v2.1.0, the Python client uses `DhanContext` for managing `client_id` and `access_token` securely.

```
from dhanhq import DhanContext, dhanhq

dhan_context = DhanContext("YOUR_CLIENT_ID","YOUR_ACCESS_TOKEN")
dhan = dhanhq(dhan_context)
```

Replace `"YOUR_CLIENT_ID"` and `"YOUR_ACCESS_TOKEN"` with your actual credentials.

Trading APIs

Orders API

The order management API allows placing, modifying, and canceling orders, as well as retrieving order and trade statuses.

Endpoints:
- `POST /orders` : Place a new order
- `PUT /orders/{order-id}` : Modify a pending order
- `DELETE /orders/{order-id}` : Cancel a pending order
- `POST /orders/slicing` : Slice order into multiple legs over freeze limit
- `GET /orders` : Retrieve the list of all orders for the day
- `GET /orders/{order-id}` : Retrieve the status of an order
- `GET /orders/external/{correlation-id}` : Retrieve the status of an order by correlation id
- `GET /trades` : Retrieve the list of all trades for the day
- `GET /trades/{order-id}` : Retrieve the details of trade by an order id

Order Placement (`POST /orders`)

Request Structure (JSON):


```
{
  "dhanClientId":"1000000003",
  "correlationId":"123abc678",
  "transactionType":"BUY",
  "exchangeSegment":"NSE_EQ",
  "productType":"INTRADAY",
  "orderType":"MARKET",
  "validity":"DAY",
  "securityId":"11536",
  "quantity":"5",
  "disclosedQuantity":"",
  "price":"",
  "triggerPrice":"",
  "afterMarketOrder":false,
  "amoTime":"",
  "boProfitValue":"",
  "boStopLossValue": ""
}
```

Parameters:

Field	Type	Description
dhanClientId required	string	User specific identification generated by Dhan
correlationId	string	The user/partner generated id for tracking back.
transactionType required	enum string	The trading side of transaction `BUY` `SELL`
exchangeSegment required	enum string	Exchange Segment of instrument to be subscribed as found in Annexure
productType required	enum string	Product type `CNC` `INTRADAY` `MARGIN` `MTF` `CO` `BO`
orderType required	enum string	Order Type `LIMIT` `MARKET` `STOP_LOSS` `STOP_LOSS_MARKET`
validity required	enum string	Validity of Order `DAY` `IOC`
securityId required	string	Exchange standard ID for each scrip. Refer Instrument List
quantity required	int	Number of shares for the order
disclosedQuantity	int	Number of shares visible (Keep more than 30% of quantity)
price required	float	Price at which order is placed
triggerPrice conditionally required	float	Price at which the order is triggered, in case of SL-M & SL-L
afterMarketOrder conditionally required	boolean	Flag for orders placed after market hours
amoTime conditionally required	enum sting	Timing to pump the after market order `PRE_OPEN` `OPEN` `OPEN_30` `OPEN_60`
boProfitValue conditionally required	float	Bracket Order Target Price change
boStopLossValue conditionally required	float	Bracket Order Stop Loss Price change

Response Structure:

```
{
  "orderId": "112111182198",
  "orderStatus": "PENDING"
}
```

Response Parameters:

Field	Type	Description
orderId	string	Order specific identification generated by Dhan
orderStatus	enum string	Last updated status of the order `TRANSIT` `PENDING` `REJECTED` `CANCELLED` `TRADED` `EXPIRED`

Python Example:

```
dhan.place_order(
  security_id=\'1333\',          # HDFC Bank
  exchange_segment=dhan.NSE,
  transaction_type=dhan.BUY,
  quantity=10,
  order_type=dhan.MARKET,
  product_type=dhan.INTRA,
  price=0
)
```

Order Modification (`PUT /orders/{order-id}`)

Request Structure (JSON):

```
{
  "dhanClientId":"1000000009",
  "orderId":"112111182045",
  "orderType":"LIMIT",
  "legName":"",
  "quantity":"40",
  "price":"3345.8",
  "disclosedQuantity":"10",
  "triggerPrice":"",
  "validity":"DAY"
}
```

Parameters:

Field	Type	description
dhanClientId required	string	User specific identification generated by Dhan
orderId required	string	Order specific identification generated by Dhan
orderType required	enum string	Order Type `LIMIT` `MARKET` `STOP_LOSS` `STOP_LOSS_MARKET`
legName conditionally required	enum string	In case of BO & CO, which leg is modified `ENTRY_LEG` `TARGET_LEG` `STOP_LOSS_LEG`
quantity conditionally required	int	Quantity to be modified
price conditionally required	float	Price to be modified
disclosedQuantity	int	Number of shares visible (if opting keep >30% of quantity)
triggerPrice conditionally required	float	Price at which the order is triggered, in case of SL-M & SL-L
validity required	enum string	Validity of Order `DAY` `IOC`

Response Structure:

```
{
  "orderId": "112111182045",
  "orderStatus": "TRANSIT"
}
```

Response Parameters:

Field	Type	Description
orderId	string	Order specific identification generated by Dhan
orderStatus	enum string	Last updated status of the order `TRANSIT` `PENDING` `REJECTED` `CANCELLED` `TRADED` `EXPIRED`

Python Example:

```
dhan.modify_order(order_id, order_type, leg_name, quantity, price, trigger_price, disclosed_quantity, validity)
```

Order Cancellation (`DELETE /orders/{order-id}`)

Request Structure: No Body

Response Structure:

```
{
  "orderId": "112111182045",
  "orderStatus": "CANCELLED"
}
```

Response Parameters:

Field	Type	Description
orderId	string	Order specific identification generated by Dhan
orderStatus	enum string	Last updated status of the order `TRANSIT` `PENDING` `REJECTED` `CANCELLED` `TRADED` `EXPIRED`

Python Example:

```
dhan.cancel_order(order_id)
```

Order Slicing (`POST /orders/slicing`)

Request Structure (JSON):

```
{
  "dhanClientId":"1000000003",
  "correlationId":"123abc678",
  "transactionType":"BUY",
  "exchangeSegment":"NSE_EQ",
  "productType":"INTRADAY",
  "orderType":"MARKET",
  "validity":"DAY",
  "securityId":"11536",
  "quantity":"5",
  "disclosedQuantity":"",
  "price":"",
  "triggerPrice":"",
  "afterMarketOrder":false,
  "amoTime":"",
  "boProfitValue":"",
  "boStopLossValue": ""
}
```


Parameters: (Same as Order Placement)

Response Structure: (Likely same as Order Placement, needs confirmation from documentation)

Retrieve All Orders (`GET /orders`)

Python Example:

```
dhan.get_order_list()
```

Retrieve Order by ID (`GET /orders/{order-id}`)

Python Example:

```
dhan.get_order_by_id(order_id)
```

Retrieve Order by Correlation ID (`GET /orders/external/{correlation-id}`)

Python Example:

```
dhan.get_order_by_corelationID(corelationID)
```

Retrieve All Trades (`GET /trades`)

Python Example:

```
dhan.get_trade_book(order_id) # Note: The Python client uses order_id for get_trade_book, while the API doc shows /trades endpoint without order_id.
```

Retrieve Trade by Order ID (`GET /trades/{order-id}`)

Python Example:

```
dhan.get_trade_history(from_date,to_date,page_number=0)
```

Super Order API

Super Order allows placing a combination of entry, target, and stop-loss orders. The Python client provides functions for managing Super Orders.

Python Examples:

```
 # Get Super Order
dhan.get_super_order(
    super_order_id=\'your_super_order_id\'
)

# Place Super Order
dhan.place_super_order(
    security_id=\'1333\',
    exchange_segment=dhan.NSE,
    transaction_type=dhan.BUY,
    quantity=10,
    order_type=dhan.LIMIT,
    product_type=dhan.INTRA,
    price=100,
    validity=dhan.DAY,
    trigger_price=0,
    amo=False,
    bo_profit_value=0,
    bo_stop_loss_value=0,
    leg_name=None
)

# Modify Super Order
dhan.modify_super_order(
    super_order_id=\'your_super_order_id\
    order_type=dhan.LIMIT,
    quantity=12,
```

```
    price=105,
    trigger_price=0,
    validity=dhan.DAY,
    bo_profit_value=12,
    bo_stop_loss_value=6,
    leg_name=None
)

# Cancel Super Order
dhan.cancel_super_order(
    super_order_id=\'your_super_order_id\'
)

# Get Super Order List
dhan.get_super_order_list()
```

Forever Order API

Users can create and manage Forever Orders or Good Till Triggered orders using this set of APIs.

Endpoints:
- `POST /forever/orders` : Create Forever Order
- `PUT /forever/orders/{order-id}` : Modify existing Forever Order
- `DELETE /forever/orders/{order-id}` : Cancel existing Forever Order
- `GET /forever/orders` : Retrieve an array of all existing forever orders

Create Forever Order (`POST /forever/orders`)

Request Structure (JSON):

```
{
  "dhanClientId": "1000000132",
  "correlationId": "",
  "orderFlag": "OCO",
  "transactionType": "BUY",
  "exchangeSegment": "NSE_EQ",
  "productType": "CNC",
  "orderType": "LIMIT",
  "validity":"DAY",
  "securityId": "1333",
  "quantity": 5,
  "disclosedQuantity": 1,
  "price": 1428,
  "triggerPrice": 1427,
  "price1": 1420,
  "triggerPrice1": 1419,
```

```
  "quantity1": 10
}
```

Parameters:

Field	Type	Description
dhanClientId required	string	User specific identification generated by Dhan
correlationId	string	The user/partner generated id for tracking back
orderFlag required	string	Order Flag OCO for OCO order and SINGLE for Forever Order `SINGLE` `OCO`
transactionType required	string	The trading side of transaction `BUY` `SELL`
exchangeSegment required	string	Exchange & Segment `NSE_EQ` `NSE_FNO` `BSE_EQ` `NSE_FNO`
productType required	string	Product type `CNC` `MTF`
orderType required	enum string	Order Type `LIMIT` `MARKET`
validity required	string	Validity of Order for execution `DAY` `IOC`
securityId required	string	Exchange standard ID for each scrip. Refer Instrument List
quantity required	int	Number of shares for the order
disclosedQuantity	int	Number of shares visible (Keep more than 30% of quantity)
price required	float	Price at which order is placed
triggerPrice required	float	Price at which order is to be triggered
price1 conditionally required	float	Target price for OCO order
triggerPrice1 conditionally required	float	Target trigger price For OCO order
quantity1 conditionally required	integer	Target Quantity for OCO order

Response Structure:

```
{
  "orderId": "5132208051112",
  "orderStatus": "PENDING"
}
```

Response Parameters:

Field	Type	Description
orderId	string	Order specific identification generated by Dhan
orderStatus	string	Order Status `TRANSIT` `PENDING` `REJECTED` `CANCELLED` `TRADED` `EXPIRED` `CONFIRM`

Python Example:

```
 dhan.place_forever_order(
    security_id=\'1333\',
    exchange_segment=dhan.NSE,
    transaction_type=dhan.BUY,
    quantity=10,
    order_type=dhan.LIMIT,
    product_type=dhan.INTRA,
    price=100,
    validity=dhan.DAY,
    trigger_price=0,
    amo=False,
    bo_profit_value=0,
    bo_stop_loss_value=0,
    leg_name=None
)

# Place Forever Order (OCO)
dhan.place_forever_order(
    security_id=\'1333\',
    exchange_segment=dhan.NSE,
    transaction_type=dhan.BUY,
    quantity=10,
    order_type=dhan.LIMIT,
    product_type=dhan.INTRA,
    price=100,
    validity=dhan.DAY,
    trigger_price=0,
    amo=False,
    bo_profit_value=10,
    bo_stop_loss_value=5,
    leg_name=None
)
```

Modify Forever Order (`PUT /forever/orders/{order-id}`)

Request Structure (JSON):

```
 {
    "dhanClientId": "1000000132",
    "orderId": "5132208051112",
    "orderFlag": "SINGLE",
    "orderType": "LIMIT",
    "legName": "TARGET_LEG",
    "quantity": 15,
    "price": 1421,
    "disclosedQuantity":1,
    "triggerPrice": 1420,
```

```
  "validity": "DAY"
}
```

Parameters:

Field	Type	Description
dhanClientId required	string	User specific identification generated by Dhan
orderId required	enum string	Order specific identification generated by Dhan
orderFlag required	string	Order Flag OCO for OCO order and SINGLE for Forever Order `SINGLE` `OCO`
orderType required	enum string	Order Type `LIMIT` `MARKET` `STOP_LOSS` `STOP_LOSS_MARKET`
legName required	string	Order leg of Forever Order where modification is to be done `ENTRY_LEG` - For Single and First leg of OCO `TARGET_LEG` - For Second leg of OCO
quantity required	int	Number of shares for the order
price required	float	Price at which order is placed
disclosedQuantity	int	Number of shares visible (Keep more than 30% of quantity)
triggerPrice required	float	Price at which order is to be triggered
validity required	string	Validity of Order for execution `DAY` `IOC`

Response Structure:

```
{
  "orderId": "5132208051112",
  "orderStatus": "PENDING"
}
```

Response Parameters:

Field	Type	Description
orderId	string	Order specific identification generated by Dhan
orderStatus	string	Last updated status of the order `TRANSIT` `PENDING` `REJECTED` `CANCELLED` `TRADED` `EXPIRED` `CONFIRM`

Python Example:

```
dhan.modify_forever_order(
  order_id=\'your_order_id\',
  order_type=dhan.LIMIT,
  quantity=12,
  price=105,
  trigger_price=0,
  validity=dhan.DAY,
```

```
    bo_profit_value=12,
    bo_stop_loss_value=6,
    leg_name=None
)
```

Delete Forever Order (`DELETE /forever/orders/{order-id}`)

Request Structure: No Body

Response Structure:

```
{
  "orderId": "5132208051112",
  "orderStatus": "CANCELLED"
}
```

Response Parameters:

Field	Type	Description
orderId	string	Order specific identification generated by Dhan
orderStatus	string	Order Status `TRANSIT` `PENDING` `REJECTED` `CANCELLED` `TRADED` `EXPIRED` `CONFIRM`

Python Example:

```
dhan.cancel_forever_order(
    order_id=\'your_order_id\'
)
```

All Forever Order Detail (`GET /forever/orders`)

Request Structure: No Body

Response Structure:

```
[
  {
    "dhanClientId": "1000000132",
    "orderId": "1132208051115",
    "orderStatus": "CONFIRM",
    "transactionType": "BUY",
    "exchangeSegment": "NSE_EQ",
    "productType": "CNC",
    "orderType": "SINGLE",
    "tradingSymbol": "HDFCBANK",
```

```
        "securityId": "1333",
        "quantity": 10,
        "price": 1428,
        "triggerPrice": 1427,
        "legName": "ENTRY_LEG",
        "createTime": "2022-08-05 12:41:19",
        "updateTime": null,
        "exchangeTime": null,
        "drvExpiryDate": null,
        "drvOptionType": null,
        "drvStrikePrice": 0
    }
]
```


Response Parameters:

Field	Type	Description
dhanClientId	string	User specific identification generated by Dhan
orderId	string	Order specific identification generated by Dhan
orderStatus	enum string	Last updated status of the order `TRANSIT` `PENDING` `REJECTED` `CANCELLED` `TRADED` `EXPIRED` `CONFIRM`
transactionType	enum string	The trading side of transaction `BUY` `SELL`
exchangeSegment	enum string	Exchange & Segment `NSE_EQ` `NSE_FNO` `BSE_EQ` `MCX_COMM`
productType	enum string	Product type of trade `CNC` `INTRADAY` `MARGIN` `CO` `BO`
orderType	enum string	Order Type `SINGLE` `OCO`
tradingSymbol	string	Symbol of the instrument in which forever order is placed
securityId	string	Exchange standard ID for each scrip. Refer Instrument List
quantity	int	Number of shares for the order
price	float	Price at which order is placed
triggerPrice	float	Price at which order is to be triggered
legName	string	Order leg of Forever Order `ENTRY_LEG` - For Single and First leg of OCO `TARGET_LEG` - For Second leg of OCO
createTime	string	Time at which the Forever Order is created
updateTime	string	Time at which the Forever Order is updated
exchangeTime	string	Time at which order reached at exchange end
drvExpiryDate	string	Contract Expiry Date for Derivatives
drvOptionType	string	Option Type for Derivatives `CALL` `PUT`
drvStrikePrice	float	Strike Price for Derivatives

Python Example:

```
dhan.get_super_order_list() # This function in the Python client retrieves all super orders, which might include forever orders.
```

Portfolio API

This API lets you retrieve holdings and positions in your portfolio.

Endpoints:
- `GET /holdings` : Retrieve list of holdings in demat account
- `GET /positions` : Retrieve open positions
- `POST /positions/convert` : Convert intraday position to delivery or delivery to intraday

Holdings (`GET /holdings`)

Request Structure: No Body

Response Structure:


```
[
  {
    "exchange": "ALL",
    "tradingSymbol": "HDFC",
    "securityId": "1330",
    "isin": "INE001A01036",
    "totalQty": 1000,
    "dpQty": 1000,
    "t1Qty": 0,
    "availableQty": 1000,
    "collateralQty": 0,
    "avgCostPrice": 2655.0
  }
]
```


Parameters:

Field	Type	Description
exchange	enum string	Exchange
tradingSymbol	string	Refer Trading Symbol at Page No
securityId	string	Exchange standard ID for each scrip. Refer Instrument List
isin	string	Universal standard ID for each scrip
totalQty	int	Total quantity
dpQty	int	Quantity delivered in demat account
t1Qty	int	Quantity pending delivered in demat account
availableQty	int	Quantity available for transaction

| collateralQty | int | Quantity placed as collateral with broker |
| avgCostPrice | float | Average Buy Price of total quantity |

Python Example:

```
dhan.get_holdings()
```

Positions (`GET /positions`)

Request Structure: No Body

Response Structure:


```
[
  {
    "dhanClientId": "1000000009",
    "tradingSymbol": "TCS",
    "securityId": "11536",
    "positionType": "LONG",
    "exchangeSegment": "NSE_EQ",
    "productType": "CNC",
    "buyAvg": 3345.8,
    "buyQty": 40,
    "costPrice": 3215.0,
    "sellAvg": 0.0,
    "sellQty": 0,
    "netQty": 40,
    "realizedProfit": 0.0,
    "unrealizedProfit": 6122.0,
    "rbiReferenceRate": 1.0,
    "multiplier": 1,
    "carryForwardBuyQty": 0,
    "carryForwardSellQty": 0,
    "carryForwardBuyValue": 0.0,
    "carryForwardSellValue": 0.0,
    "dayBuyQty": 40,
    "daySellQty": 0,
    "dayBuyValue": 133832.0,
    "daySellValue": 0.0,
    "drvExpiryDate": "0001-01-01",
    "drvOptionType": null,
    "drvStrikePrice": 0.0,
    "crossCurrency": false
  }
]
```


Parameters:
| Field | Type | Description |

| --- | --- | --- |
dhanClientId	string	User specific identification generated by Dhan
tradingSymbol	string	Refer Trading Symbol in Tables
securityId	string	Exchange standard id for each scrip. Refer Instrument List
positionType	enum string	Position Type `LONG` `SHORT` `CLOSED`
exchangeSegment	enum string	Exchange & Segment `NSE_EQ` `NSE_FNO` `NSE_CURRENCY` `BSE_EQ` `BSE_FNO` `BSE_CURRENCY` `MCX_COMM`
productType	enum string	Product type `CNC` `INTRADAY` `MARGIN` `MTF` `CO` `BO`
buyAvg	float	Average buy price mark to market
buyQty	int	Total quantity bought
costPrice	int	Actual Cost Price
sellAvg	float	Average sell price mark to market
sellQty	int	Total quantities sold
netQty	int	buyQty - sellQty = netQty
realizedProfit	float	Profit or loss booked
unrealizedProfit	float	Profit or loss standing for open position
rbiReferenceRate	float	RBI mandated reference rate for forex
multiplier	int	Multiplier
carryForwardBuyQty	int	Carry forward buy quantity
carryForwardSellQty	int	Carry forward sell quantity
carryForwardBuyValue	float	Carry forward buy value
carryForwardSellValue	float	Carry forward sell value
dayBuyQty	int	Day buy quantity
daySellQty	int	Day sell quantity
dayBuyValue	float	Day buy value
daySellValue	float	Day sell value
drvExpiryDate	string	Derivative expiry date
drvOptionType	string	Derivative option type
drvStrikePrice	float	Derivative strike price
crossCurrency	boolean	Cross currency flag

Python Example:

```
dhan.get_positions()
```

Convert Position (`POST /positions/convert`)

Python Example: (No direct example in GitHub, but the API exists)

EDIS API

To sell holding stocks, one needs to complete the CDSL eDIS flow, generate T-PIN & mark stock to complete the sell action.

Python Examples:

```
 # Generate TPIN
dhan.generate_tpin()

# Enter TPIN in Form
dhan.open_browser_for_tpin(
    isin=\'INE00IN01015\',
    qty=1,
    exchange=\'NSE\'
)

# EDIS Status and Inquiry
dhan.edis_inquiry()
```

Trader's Control API (Kill Switch)

This API lets you activate or deactivate the kill switch for your account, which will disable trading for the current trading day.

Endpoint:
- `POST /killswitch` : Activate/Deactivate Kill Switch

Request Structure (URL Parameters):
`https://api.dhan.co/v2/killswitch?killSwitchStatus=ACTIVATE` (or `DEACTIVATE`)

Request Headers:
- `Accept: application/json`
- `Content-Type: application/json`
- `access-token: JWT`

Request Body: No Body

Response Structure:

```
 {
    "dhanClientId":"1000000009",
    "killSwitchStatus": "Kill Switch has been successfully activated"
}
```

Response Parameters:

Field	Type	Description
dhanClientId	string	User specific identification generated by Dhan
killSwitchStatus	string	Status of Kill Switch - activated or not

Note: You need to ensure that all your positions are closed and there are no pending orders in your account to be able to activate Kill Switch.

Python Example:

```
dhan.kill_switch(
  kill_switch_status=dhan.ACTIVATE
)
```

Funds API

Users can get details about the fund requirements or available funds (with margin requirements) in their Trading Account.

Endpoints:

- `POST /margincalculator` : Margin requirement for any order
- `GET /fundlimit` : Retrieve trading account fund information

Margin Calculator (`POST /margincalculator`)

Request Structure (JSON):

```
{
  "dhanClientId": "1000000132",
  "exchangeSegment": "NSE_EQ",
  "transactionType": "BUY",
  "quantity": 5,
  "productType": "CNC",
  "securityId": "1333",
  "price": 1428
}
```

Parameters:

Field	Type	Description
dhanClientId	string	User specific identification generated by Dhan
exchangeSegment	string	Exchange Segment of instrument to be subscribed as found in Annexure

transactionType	string	The trading side of transaction `BUY` `SELL`
quantity	int	Number of shares for the order
productType	string	Product type `CNC` `INTRADAY` `MARGIN` `MTF` `CO` `BO`
securityId	string	Exchange standard ID for each scrip. Refer Instrument List
price	float	Price at which order is placed

Response Structure:


```
{
  "dhanClientId": "1000000132",
  "marginRequired": 1000.00,
  "adhocMargin": 0.00,
  "spanMargin": 0.00,
  "exposureMargin": 0.00,
  "optionPremium": 0.00,
  "varMargin": 0.00,
  "premiumMargin": 0.00,
  "brokerage": 0.00,
  "totalTax": 0.00,
  "totalCharges": 0.00,
  "leverage": 0.00
}
```


Response Parameters:

Field	Type	Description
dhanClientId	string	User specific identification generated by Dhan
marginRequired	float	Total margin required for the order
adhocMargin	float	Adhoc margin
spanMargin	float	SPAN margin
exposureMargin	float	Exposure margin
optionPremium	float	Option premium
varMargin	float	VAR margin
premiumMargin	float	Premium margin
brokerage	float	Brokerage
totalTax	float	Total tax
totalCharges	float	Total charges
leverage	float	Leverage

Fund Limit (`GET /fundlimit`)

Request Structure: No Body

Response Structure:

```
{
  "dhanClientId": "1000000132",
  "availableBalance": 100000.00,
  "withdrawableBalance": 90000.00,
  "cashMargin": 80000.00,
  "collateralMargin": 10000.00,
  "intradayPayin": 5000.00,
  "approvedPayin": 5000.00,
  "pendingPayin": 0.00,
  "utilizedMargin": 10000.00,
  "m2mUnrealized": 500.00,
  "m2mRealized": 200.00,
  "holdingSales": 1000.00,
  "adhocMargin": 0.00,
  "spanMargin": 0.00,
  "exposureMargin": 0.00,
  "optionPremium": 0.00,
  "varMargin": 0.00,
  "premiumMargin": 0.00,
  "brokerage": 0.00,
  "totalTax": 0.00,
  "totalCharges": 0.00,
  "leverage": 0.00
}
```


Response Parameters:

Field	Type	Description
dhanClientId	string	User specific identification generated by Dhan
availableBalance	float	Available balance for trading
withdrawableBalance	float	Withdrawable balance
cashMargin	float	Cash margin
collateralMargin	float	Collateral margin
intradayPayin	float	Intraday payin
approvedPayin	float	Approved payin
pendingPayin	float	Pending payin
utilizedMargin	float	Utilized margin
m2mUnrealized	float	Mark to market unrealized profit/loss
m2mRealized	float	Mark to market realized profit/loss
holdingSales	float	Holding sales
adhocMargin	float	Adhoc margin
spanMargin	float	SPAN margin
exposureMargin	float	Exposure margin
optionPremium	float	Option premium
varMargin	float	VAR margin

premiumMargin	float	Premium margin
brokerage	float	Brokerage
totalTax	float	Total tax
totalCharges	float	Total charges
leverage	float	Leverage

Python Example:

```
dhan.get_fund_limits()
```

Statement API

This set of APIs retrieves all your trade and ledger details to help you summarize and analyze your trades.

Endpoints:
- `GET /ledger` : Retrieve Trading Account debit and credit details
- `GET /trades/` : Retrieve historical trade data

Ledger Report (`GET /ledger`)

Query Parameters:
Field	Description
from-date required	Date from which Ledger Report is required in format `YYYY-MM-DD`
to-date required	Date upto which Ledger Report is required in format `YYYY-MM-DD`

Request Structure: No Body

Response Structure:


```
{
  "dhanClientId": "1000000001",
  "narration": "FUNDS WITHDRAWAL",
  "voucherdate": "Jun 22, 2022",
  "exchange": "NSE-CAPITAL",
  "voucherdesc": "PAYBNK",
  "vouchernumber": "202200036701",
  "debit": "20000.00",
  "credit": "0.00",
  "runbal": "957.29"
}
```

Parameters:

Field	Type	Description
dhanClientId	string	User specific identification generated by Dhan
narration	string	Description of the ledger transaction
voucherdate	string	Transaction Date
exchange	string	Exchange information for the transaction
voucherdesc	string	Nature of transaction
vouchernumber	string	System generated transaction number
debit	string	Debit amount (only when credit returns 0)
credit	string	Credit amount (only when debit returns 0)
runbal	string	Running Balance post transaction

Trade History (`GET /trades/{from-date}/{to-date}/{page}`)

Path Parameters:

Field	Description
from-date required	Date from which Trade History is required in format `YYYY-MM-DD`
to-date required	Date upto which Trade History is required in format `YYYY-MM-DD`
page required	Page number of which data is being fetched. Pass `0` as default.

Request Structure: No Body

Response Structure:


```
[
  {
    "dhanClientId": "1000000001",
    "orderId": "212212307731",
    "exchangeOrderId": "76036896",
    "exchangeTradeId": "407958",
    "transactionType": "BUY",
    "exchangeSegment": "NSE_EQ",
    "productType": "CNC",
    "orderType": "MARKET",
    "tradingSymbol": null,
    "customSymbol": "Tata Motors",
    "securityId": "3456",
    "tradedQuantity": 1,
    "tradedPrice": 390.9,
    "isin": "INE155A01022",
    "instrument": "EQUITY",
    "sebiTax": 0.0004,
    "stt": 0,
```

```
        "brokerageCharges": 0,
        "serviceTax": 0.0025,
        "exchangeTransactionCharges": 0.0135,
        "stampDuty": 0,
        "createTime": "NA",
        "updateTime": "NA",
        "exchangeTime": "2022-12-30 10:00:46",
        "drvExpiryDate": "NA",
        "drvOptionType": "NA",
        "drvStrikePrice": 0
    }
]
```


Parameters:

Field	Type	Description
dhanClientId	string	User specific identification generated by Dhan
orderId	string	Order specific identification generated by Dhan
exchangeOrderId	string	Order specific identification generated by exchange
exchangeTradeId	string	Trade specific identification generated by exchange
transactionType	enum string	The trading side of transaction `BUY` `SELL`
exchangeSegment	enum string	Exchange Segment of instrument to be subscribed as found in Annexure
productType	enum string	Product type of trade `CNC` `INTRADAY` `MARGIN` `MTF` `CO` `BO`
orderType	enum string	Order Type `LIMIT` `MARKET` `STOP_LOSS` `STOP_LOSS_MARKET`
tradingSymbol	string	Refer Trading Symbol in Tables
customSymbol	string	Trading Symbol as per Dhan
securityId	string	Exchange standard ID for each scrip. Refer Instrument List
tradedQuantity	int	Number of shares executed
tradedPrice	float	Price at which trade is executed
isin	string	Universal standard ID for each scrip
instrument	string	Type of Instrument `EQUITY` `DERIVATIVES`
sebiTax	string	SEBI Turnover Charges
stt	string	Securities Transactions Tax
brokerageCharges	string	Brokerage charges by Dhan, refer pricing here
serviceTax	string	Applicable Service Tax
exchangeTransactionCharges	string	Exchange Transaction Charge
stampDuty	string	Stamp Duty Charges
createTime	string	Time at which the order is created
updateTime	string	Time at which the last activity happened
exchangeTime	string	Time at which order reached at exchange

drvExpiryDate	int	For F&O, expiry date of contract
drvOptionType	enum string	Type of Option `CALL` `PUT`
drvStrikePrice	float	For Options, Strike Price

Python Example:

```
dhan.get_trade_history(from_date,to_date,page_number=0)
```

Postback API

Postback API or Webhooks uses a `POST` request with **JSON payload** to the Postback URL. This JSON payload contains order updates in case of status changes or modifications.

Postback Payload Structure:


```
{
  "dhanClientId": "1000000003",
  "orderId": "112111182198",
  "correlationId":"123abc678",
  "orderStatus": "PENDING",
  "transactionType": "BUY",
  "exchangeSegment": "NSE_EQ",
  "productType": "INTRADAY",
  "orderType": "MARKET",
  "validity": "DAY",
  "tradingSymbol": "",
  "securityId": "11536",
  "quantity": 5,
  "disclosedQuantity": 0,
  "price": 0.0,
  "triggerPrice": 0.0,
  "afterMarketOrder": false,
  "boProfitValue": 0.0,
  "boStopLossValue": 0.0,
  "legName": null,
  "createTime": "2021-11-24 13:33:03",
  "updateTime": "2021-11-24 13:33:03",
  "exchangeTime": "2021-11-24 13:33:03",
  "drvExpiryDate": null,
  "drvOptionType": null,
  "drvStrikePrice": 0.0,
  "omsErrorCode": null,
  "omsErrorDescription": null
}
```

Parameters:

Field	Type	Description
dhanClientId	string	User specific identification generated by Dhan
orderId	string	Order specific identification generated by Dhan
correlationId	string	The user/partner generated id for tracking back
orderStatus	enum string	Last updated status of the order `TRANSIT` `PENDING` `REJECTED` `CANCELLED` `TRADED` `EXPIRED`
transactionType	enum string	The trading side of transaction `BUY` `SELL`
exchangeSegment	enum string	Exchange & Segment `NSE_EQ` `NSE_FNO` `NSE_CURRENCY` `BSE_EQ` `MCX_COMM`
productType	enum string	Product type of trade `CNC` `INTRADAY` `MARGIN` `MTF` `CO` `BO`
orderType	enum string	Order Type `LIMIT` `MARKET` `STOP_LOSS` `STOP_LOSS_MARKET`
validity	enum string	Validity of Order `DAY` `IOC`
tradingSymbol	string	Refer Trading Symbol in Tables
securityId	string	Exchange standard id for each scrip. Refer Instrument List
quantity	int	Number of shares for the order
disclosedQuantity	int	Number of shares visible
price	float	Price at which order is placed
triggerPrice	float	Price at which order is triggered, for SL-M, SL-L, CO & BO
afterMarketOrder	boolean	The order placed is AMO ?
boProfitValue	float	Bracket Order Target Price change
boStopLossValue	float	Bracket Order Stop Loss Price change
legName	enum string	Leg identification in case of BO `ENTRY_LEG` `TARGET_LEG` `STOP_LOSS_LEG`
createTime	string	Time at which the order is created
updateTime	string	Time at which the last activity happened
exchangeTime	string	Time at which order reached at exchange
drvExpiryDate	int	For F&O, expiry date of contract
drvOptionType	enum string	Type of Option `CALL` `PUT`
drvStrikePrice	float	For Options, Strike Price
omserroeCode	string	Error code in case the order is rejected or failed
omsErrorDescription	string	Description of error in case the order is rejected or failed

Setting up Postback:

To set up Postback API, you will need to provide a unique Postback URL to receive callbacks. This is done by entering your URL in the 'Postback URL' field while generating the access token on web.dhan.co.

Important: You will not receive postback calls if Postback URL is set to `localhost` or `127.0.0.1`.

Live Order Update API

Realtime order updates of all your orders can be received directly via WebSocket in your system. The messages sent over this WebSocket will be JSON.

WebSocket Endpoint: `wss://api-order-update.dhan.co`

Establishing Connection

While establishing connection, you need to send an Authorisation Message for connection.

For Individual Users:

Authorisation message structure:

```
{
  "LoginReq":{
    "MsgCode": 42,
    "ClientId":"1000000001",
    "Token":"JWT"
  },
  "UserType": "SELF"
}
```

Parameters:

Field	Type	Description
LoginReq required	object	JSON for adding Client ID and Access Token
MsgCode required	int	Message Code for getting Order Updates `42` by default
ClientId required	string	User specific identification generated by Dhan
Token required	string	Access Token generated for user
UserType required	string	`SELF` for individual users

For Partners:

Authorisation message structure:

```
{
  "LoginReq":{
    "MsgCode": 42,
    "ClientId": "partner_id"
```

```
    },
    "UserType": "PARTNER",
    "Secret": "partner_secret"
}
```

Parameters:

Field	Type	Description
LoginReq required	object	JSON for adding Client ID and Access Token
MsgCode required	int	Message Code for getting Order Updates `42` by default
ClientId required	string	`partner_id` generated for the partner
UserType required	string	`PARTNER` for partner platforms
Secret required	string	`partner_secret` generated for the partner

Order Update Message Structure:


```
{
  "Data": {
    "series": "EQ",
    "goodTillDaysDate": "2024-09-11",
    "instrumentType": "EQ",
    "refLtp": 13.21,
    "tickSize": 0.01,
    "algoId": "0",
    "multiplier": 1,
    "Exchange": "NSE",
    "Segment": "E",
    "Source": "N",
    "SecurityId": "14366",
    "ClientId": "1000000001",
    "ExchOrderNo": "1400000000404591",
    "OrderNo": "1124091136546",
    "Product": "C",
    "TxnType": "B",
    "OrderType": "LMT",
    "Validity": "DAY",
    "DiscQuantity": 1,
    "DiscQtyRem": 1,
    "RemainingQuantity": 1,
    "Quantity": 1,
    "TradedQty": 0,
    "Price": 13,
    "TriggerPrice": 0,
    "TradedPrice": 0,
    "AvgTradedPrice": 0,
    "AlgoOrdNo": null,
    "OffMktFlag": "0",
    "OrderDateTime": "2024-09-11 14:39:29",
```

```
        "ExchOrderTime": "2024-09-11 14:39:29",
        "LastUpdatedTime": "2024-09-11 14:39:29",
        "Remarks": "NR",
        "MktType": "NL",
        "ReasonDescription": "CONFIRMED",
        "LegNo": 1,
        "Instrument": "EQUITY",
        "Symbol": "IDEA",
        "ProductName": "CNC",
        "Status": "Cancelled",
        "LotSize": 1,
        "StrikePrice": null,
        "ExpiryDate": "0001-01-01 00:00:00",
        "OptType": "XX",
        "DisplayName": "Vodafone Idea",
        "Isin": "INE669E01016",
        "Series": "EQ",
        "GoodTillDaysDate": "2024-09-11",
        "RefLtp": 13.21,
        "TickSize": 0.01,
        "AlgoId": "0",
        "Multiplier": 1,
        "CorrelationId": "",
        "Remarks": "Super Order"
    },
    "Type": "order_alert"
}
```


Parameters:

Field	Type	Description
Exchange	string	Exchange in which order is placed
Segment	string	Segment for which order is placed
Source	string	Platform via which order is placed - `P` for API Orders
SecurityId	string	Exchange standard ID for each scrip. Refer Instrument List
ClientId	string	User specific identification generated by Dhan
ExchOrderNo	string	Order specific identification generated by Exchange
OrderNo	string	Order specific identification generated by Dhan
Product	enum string	Product type of trade `C` for CNC, `I` for INTRADAY, `M` for MARGIN, `F` for MTF , `V` for CO, `B` for BO
TxnType	enum string	The trading side of transaction `B` for Buy `S` for Sell
OrderType	enum string	Order Type `LMT` for Limit `MKT` for Market `SL` for Stop Loss `SLM` for Stop Loss
Validity	enum string	Validity of Order `DAY` `IOC`
DiscQuantity	int	Number of shares visible
DiscQtyRem	int	Disclosed quantity pending for execution

RemainingQuantity	int	Quantity pending for execution
Quantity	int	Total order quantity placed
TradedQty	int	Actual quantity executed on exchange
Price	float	Price at which order is placed
TriggerPrice	float	Price at which order is triggered, for SL-M, SL-L, CO & BO
TradedPrice	float	Price at which trade of an order is executed
AvgTradedPrice	float	Average trade price of an order (this will be different from `Traded Price` in case of partial execution)
AlgoOrdNo	float	Entry leg order number to track Target and Stop Loss leg (in case of `BO` and `CO`)
OffMktFlag	string	`1` in case of AMO order else `0`
OrderDateTime	string	Time at which the order is received by Dhan
ExchOrderTime	string	Time at which order is placed on Exchange
LastUpdatedTime	string	Last update time of any order modification or trade
Remarks	string	Additional remarks sent along while placing order
MktType	string	`NL` for Normal Market `AU`, `A1` and `A2` for Auction Market
ReasonDescription	string	Order rejection reason
LegNo	int	`1` for Entry Leg `2` for Stop Loss Leg `3` for Target Leg
Instrument	string	Instrument in which order is placed - Instrument List
Symbol	string	Symbol in which order is placed - Refer Instrument List
ProductName	string	Product type of the order placed - Product Type
Status	enum string	Last updated status of the order `TRANSIT` `PENDING` `REJECTED` `CANCELLED` `TRADED` `EXPIRED`
LotSize	int	Lot Size in case of Derivatives
StrikePrice	float	Strike Price in which order is placed in Option contract
ExpiryDate	string	Expiry Date of the contract in which order is placed
OptType	string	`CE` or `PE` in case of Option contract
DisplayName	string	Name of instrument in which order is placed - Refer Instrument List
Isin	string	ISIN of the instrument in which order is placed
Series	string	Exchange series of the instrument
GoodTillDaysDate	string	Order validity in case of Forever Order
RefLtp	float	LTP at time of order update
TickSize	float	Tick size of the instrument
AlgoId	string	Exchange ID for special order types
Multiplier	int	In case of commodity and currency contracts
CorrelationId	string	The user/partner generated id for tracking back
Remarks	string	`Super Order` if the order is part of super order

Python Example:

```
# Live Order Update (WebSocket)
# This requires a separate WebSocket connection setup as detailed in the
documentation.
```

Data APIs

Market Quote API

This API gives you snapshots of multiple instruments at once. You can fetch LTP, Quote or Market Depth of instruments via API which sends real time data at the time of API request.

Endpoints:
- `POST /marketfeed/ltp` : Get ticker data of instruments
- `POST /marketfeed/ohlc` : Get OHLC data of instruments
- `POST /marketfeed/quote` : Get market depth data of instruments

Info: You can fetch upto 1000 instruments in single API request with rate limit of 1 request per second.

Ticker Data (`POST /marketfeed/ltp`)

Headers:
Header	Description
access-token required	Access Token generated via Dhan
client-id required	User specific identification generated by Dhan

Request Structure (JSON):

```
{
  "NSE_EQ":[11536],
  "NSE_FNO":[49081,49082]
}
```

Parameters:
Field	Field Type	Description
Exchange Segment ENUM required	array	Security ID - can be found in Instrument List

Response Structure:

```
{
  "data": {
    "NSE_EQ": {
      "11536": {
        "last_price": 4520
      }
    },
    "NSE_FNO": {
      "49081": {
        "last_price": 368.15
      },
      "49082": {
        "last_price": 694.35
      }
    }
  },
  "status": "success"
}
```


Response Parameters:

Field	Type	Description
last_price	float	LTP of the Instrument

Python Example:

```
dhan.ohlc_data(
  securities = {"NSE_EQ":[1333]} # This function can be used for ticker data as well.
)
```

OHLC Data (`POST /marketfeed/ohlc`)

Headers:

Header	Description
access-token required	Access Token generated via Dhan
client-id required	User specific identification generated by Dhan

Request Structure (JSON):

```
{
  "NSE_EQ":[11536],
  "NSE_FNO":[49081,49082]
}
```

Parameters:

Field	Field Type	Description
Exchange Segment ENUM required	array	Security ID - can be found in Instrument List

Response Structure:


```
{
  "data": {
    "NSE_EQ": {
      "11536": {
        "last_price": 4525.55,
        "ohlc": {
          "open": 4521.45,
          "close": 4507.85,
          "high": 4530,
          "low": 4500
        }
      }
    },
    "NSE_FNO": {
      "49081": {
        "last_price": 368.15,
        "ohlc": {
          "open": 0,
          "close": 368.15,
          "high": 0,
          "low": 0
        }
      },
      "49082": {
        "last_price": 694.35,
        "ohlc": {
          "open": 0,
          "close": 694.35,
          "high": 0,
          "low": 0
        }
      }
    }
  },
  "status": "success"
}
```


Response Parameters:

Field	Type	Description

last_price	float	LTP of the Instrument
ohlc.open	float	Market opening price of the day
ohlc.close	float	Market closing price of the day
ohlc.high	float	Day High price
ohlc.low	float	Day Low price

Python Example:

```
dhan.ohlc_data(
  securities = {"NSE_EQ":[1333]}
)
```

Market Depth Data (`POST /marketfeed/quote`)

Headers:
Header	Description
access-token required	Access Token generated via Dhan
client-id required	User specific identification generated by Dhan

Request Structure (JSON):

```
{
  "NSE_FNO":[49081]
}
```

Parameters:
Field	Field Type	Description
Exchange Segment ENUM required	array	Security ID - can be found in Instrument List

Response Structure:

```
{
  "data": {
    "NSE_FNO": {
      "49081": {
        "average_price": 0,
        "buy_quantity": 1825,
        "depth": {
          "buy": [
            {
              "quantity": 1800,
```

```
            "orders": 1,
            "price": 77
          },
          {
            "quantity": 25,
            "orders": 1,
            "price": 50
          },
          {
            "quantity": 0,
            "orders": 0,
            "price": 0
          },
          {
            "quantity": 0,
            "orders": 0,
            "price": 0
          },
          {
            "quantity": 0,
            "orders": 0,
            "price": 0
          }
        ],
        "sell": [
          {
            "quantity": 0,
            "orders": 0,
            "price": 0
          },
          {
            "quantity": 0,
            "orders": 0,
            "price": 0
          },
          {
            "quantity": 0,
            "orders": 0,
            "price": 0
          },
          {
            "quantity": 0,
            "orders": 0,
            "price": 0
          },
          {
            "quantity": 0,
            "orders": 0,
            "price": 0
          }
        ]
      },
```

```
            "last_price": 368.15,
            "last_quantity": 0,
            "last_trade_time": "01/01/1980 00:00:00",
            "lower_circuit_limit": 48.25,
            "net_change": 0,
            "ohlc": {
              "open": 0,
              "close": 368.15,
              "high": 0,
              "low": 0
            },
            "oi": 0,
            "oi_day_high": 0,
            "oi_day_low": 0,
            "sell_quantity": 0,
            "upper_circuit_limit": 510.85,
            "volume": 0
          }
        }
    },
    "status": "success"
}
```


Response Parameters:

Field	Type	Description
average_price	float	Volume weighted average price of the day
buy_quantity	int	Total buy order quantity pending at the exchange
sell_quantity	int	Total sell order quantity pending at the exchange
depth.buy.quantity	int	Number of quantity at this price depth
depth.buy.orders	int	Number of open BUY orders at this price depth
depth.buy.price	float	Price at which the BUY depth stands
depth.sell.quantity	int	Number of quantity at this price depth
depth.sell.orders	int	Number of open SELL orders at this price depth
depth.sell.price	float	Price at which the SELL depth stands
last_price	float	LTP of the Instrument
last_quantity	int	Last traded quantity
last_trade_time	string	Last trade time
lower_circuit_limit	float	Lower circuit limit
net_change	float	Net change in price
ohlc.open	float	Market opening price of the day
ohlc.close	float	Market closing price of the day
ohlc.high	float	Day High price
ohlc.low	float	Day Low price
oi	int	Open Interest

oi_day_high	int	Open Interest Day High
oi_day_low	int	Open Interest Day Low
volume	int	Total traded volume

Live Market Feed API

Real-time Market Data across exchanges and segments can now be availed on your system via WebSocket. WebSocket keeps a persistent connection open, allowing the server to push real-time data to your systems.

WebSocket Endpoint: `wss://api-feed.dhan.co?version=2&token=eyxxxxx&clientId=100xxxxxxx&authType=2`

Query Parameters for Connection:
Field	Description
version required	`2` for DhanHQ v2
token required	Access Token generated via Dhan
clientId required	User specific identification generated by Dhan
authType required	`2` by Default

Establishing Connection

While establishing connection, you need to send an Authorisation Message for connection.

For Individual Users:

Authorisation message structure:

```
{
  "LoginReq":{
    "MsgCode": 42,
    "ClientId":"1000000001",
    "Token":"JWT"
  },
  "UserType": "SELF"
}
```

Parameters:
Field	Type	Description
LoginReq required	object	JSON for adding Client ID and Access Token
MsgCode required	int	Message Code for getting Order Updates `42` by default

ClientId required	string	User specific identification generated by Dhan
Token required	string	Access Token generated for user
UserType required	string	`SELF` for individual users

For Partners:

Authorisation message structure:

```
{
  "LoginReq":{
    "MsgCode": 42,
    "ClientId": "partner_id"
  },
  "UserType": "PARTNER",
  "Secret": "partner_secret"
}
```

Parameters:
Field	Type	Description
LoginReq required	object	JSON for adding Client ID and Access Token
MsgCode required	int	Message Code for getting Order Updates `42` by default
ClientId required	string	`partner_id` generated for the partner
UserType required	string	`PARTNER` for partner platforms
Secret required	string	`partner_secret` generated for the partner

Adding Instruments

You can subscribe upto 5000 instruments in a single connection and receive market data packets. You can send multiple messages over a single connection to subscribe to all instruments and receive data (max 100 instruments per message).

Request Structure (JSON):

```
{
  "RequestCode" : 15,
  "InstrumentCount" : 2,
  "InstrumentList" : [
    {
      "ExchangeSegment" : "NSE_EQ",
      "SecurityId" : "1333"
    },
    {
      "ExchangeSegment" : "BSE_EQ",
      "SecurityId" : "532540"
    }
```

```
  ]
}
```

Parameters:
Field	Type	Description
RequestCode required	int	Code for subscribing to particular data mode. Refer to Feed Request Code to subscribe to required data mode
InstrumentCount required	int	No. of instruments to subscribe from this request
InstrumentList.ExchangeSegment required	enum string	Exchange Segment of instrument to be subscribed as found in Annexure
InstrumentList.SecurityId required	string	Exchange standard ID for each scrip. Refer Instrument List

Keeping Connection Alive

To keep the WebSocket connection alive, the server sends **Ping-Pong** messages every 10 seconds. If the client does not respond for more than 40 seconds, the connection is closed.

Market Data

The market feed data is sent as structured binary packets. All responses from Dhan Market Feed consists of Response Header and Payload.

Response Header (8 bytes):
Bytes	Type	Size	Description
`1`	[ ] byte	`1`	Feed Response Code can be referred in Annexure
`2-3`	int16	`2`	Message Length of the entire payload packet
`4`	[ ] byte	`1`	Exchange Segment can be referred in Annexure
`5-8`	int32	`4`	Security ID - can be found in Instrument List

Ticker Packet:
This packet consists of Last Traded Price (LTP) and Last Traded Time (LTT) data.

Bytes	Type	Size	Description
`0-8`	[ ] array	`8`	Response Header with code `2`
`9-12`	float32	`4`	Last Traded Price of the subscribed instrument
`13-16`	int32	`4`	Last Trade Time

Prev Close Packet:
This packet provides Previous Day data for comparison.

Bytes	Type	Size	Description
`0-8`	[ ] array	`8`	Response Header with code `6`
`9-12`	float32	`4`	Previous day closing price
`13-16`	int32	`4`	Open Interest - previous day

Quote Packet:
This packet contains complete trade data, LTP, and other information.

Bytes	Type	Size	Description
`0-8`	[ ] array	`8`	Response Header with code `4`
`9-12`	float32	`4`	Latest Traded Price of the subscribed instrument
`13-14`	int16	`2`	Last Traded Quantity
`15-18`	int32	`4`	Last Trade Time (LTT)
`19-22`	float32	`4`	Average Trade Price (ATP)
`23-26`	int32	`4`	Volume
`27-30`	int32	`4`	Total Sell Quantity
`31-34`	int32	`4`	Total Buy Quantity
`35-38`	float32	`4`	Day Open Value
`39-42`	float32	`4`	Day Close Value - only sent post market close
`43-46`	float32	`4`	Day High Value
`47-50`	float32	`4`	Day Low Value

OI Data Packet:
Open Interest (OI) data for Derivative Contracts.

Bytes	Type	Size	Description
`0-8`	[ ] array	`8`	Response Header with code `5`
`9-12`	int32	`4`	Open Interest of the contract

Full Packet:
This packet contains complete trade data, Market Depth, and OI data in a single packet.

Bytes	Type	Size	Description
`0-8`	[ ] array	`8`	Response Header with code `8`
`9-12`	float32	`4`	Latest Traded Price of the subscribed instrument

13-14	int16	2	Last Traded Quantity
15-18	int32	4	Last Trade Time (LTT)
19-22	float32	4	Average Trade Price (ATP)
23-26	int32	4	Volume
27-30	int32	4	Total Sell Quantity
31-34	int32	4	Total Buy Quantity
35-38	int32	4	Open Interest in the contract (for Derivatives)
39-42	int32	4	Highest Open Interest for the day (only for NSE_FNO)
43-46	int32	4	Lowest Open Interest for the day (only for NSE_FNO)
47-50	float32	4	Day Open Value
51-54	float32	4	Day Close Value - only sent post market close
55-58	float32	4	Day High Value
59-62	float32	4	Day Low Value
63-162	Market Depth Structure	100	5 packets of 20 bytes each for each instrument

Python Example:

```
 # Live Market Feed (WebSocket)
# This requires a separate WebSocket connection setup as detailed in the
documentation.
```

20 Market Depth API

Level 3 data includes market depth upto 20 levels, streamed real-time via websockets. This data can be used to detect demand supply zones.

Note: Only NSE Equity and Derivatives segments are enabled for 20 Level Market Depth.

WebSocket Endpoint: `wss://depth-api-feed.dhan.co/twentydepth?token=eyxxxxx&clientId=100xxxxxxx&authType=2`

Query Parameters for Connection:

Field	Description
token required	Access Token generated via Dhan
clientId required	User specific identification generated by Dhan
authType required	2 by Default

Establishing Connection

(Same as Live Market Feed API)

Adding Instruments

You can subscribe upto 50 instruments in a single connection and receive market data packets. You can send all 50 instruments in a single JSON message or multiple messages.

Request Structure (JSON):

```
{
  "RequestCode" : 23,
  "InstrumentCount" : 2,
  "InstrumentList" : [
    {
      "ExchangeSegment" : "NSE_EQ",
      "SecurityId" : "1333"
    },
    {
      "ExchangeSegment" : "NSE_FNO",
      "SecurityId" : "532540"
    }
  ]
}
```

Parameters:

Field	Type	Description
RequestCode required	int	Code for subscribing to particular data mode. `23` for 20 Level Market Depth.
InstrumentCount required	int	No. of instruments to subscribe from this request
InstrumentList.ExchangeSegment required	enum string	Exchange Segment of instrument to be subscribed as found in Annexure
InstrumentList.SecurityId required	string	Exchange standard ID for each scrip. Refer Instrument List

Keeping Connection Alive

To keep the WebSocket connection alive, the server sends **Ping-Pong** messages every 10 seconds. If the client does not respond for more than 40 seconds, the connection is closed.

20-Level Depth Packet

The market depth data is sent as structured binary packet. All responses from Dhan Market Feed consists of Response Header and Payload.

Response Header (12 bytes):

Bytes	Type	Size	Description
1-2	int16	2	Message Length of the entire payload packet
3	[ ] byte	1	Feed Response Code can be referred in Annexure
4	[ ] byte	1	Exchange Segment can be referred in Annexure
5-8	int32	4	Security ID - can be found in Instrument List
9-12	uint32	4	Message Sequence (to be ignored)

Depth Packet:

Depth Data Packet for 20 level market depth is structured differently from 5 level depth. You will receive the bid (sell) and ask (buy) data packets separately, each containing 20 packets of 16 bytes each.

Bytes	Type	Size	Description
0-12	[ ] array	12	Response Header 41 for Bid Data (Buy) 51 for Ask Data (Sell)
13-332	Bid/Ask Depth Structure	320	20 packets of 16 bytes each for each instrument

Each of these 20 packets will be received in the following packet structure:

Bytes	Type	Size	Description
1-8	float64	8	Price
9-12	uint32	4	Quantity
13-16	uint32	4	No. of Orders

Feed Disconnect:

To disconnect WebSocket, you can send the following JSON request message:

```
{
  "RequestCode" : 12
}
```

In case of WebSocket disconnection from server side, you will receive a disconnection packet with a reason code.

Note: If more than 5 websockets are established, the first socket will be disconnected with 805 with every additional connection.

Disconnection Packet Structure:

| Bytes | Type | Size | Description |

| --- | --- | --- |
| `0-12` | [ ] array | `8` | Response Header with code `50` |
| `13-14` | int16 | `2` | Disconnection message code |

Historical Data API

This API gives you historical candle data for the desired scrip across segments & exchange. This data is presented in the form of a candle and gives you timestamp, open, high, low, close & volume.

Endpoints:
- `POST /charts/historical` : Get OHLC for daily timeframe
- `POST /charts/intraday` : Get OHLC for minute timeframe

Daily Historical Data (`POST /charts/historical`)

Request Structure (JSON):

```
{
  "securityId": "1333",
  "exchangeSegment":"NSE_EQ",
  "instrument": "EQUITY",
  "expiryCode": 0,
  "oi": false,
  "fromDate": "2022-01-08",
  "toDate": "2022-02-08"
}
```

Parameters:
Field	Field Type	Description
securityId required	string	Exchange standard ID for each scrip. Refer Instrument List
exchangeSegment required	enum string	Exchange & segment for which data is to be fetched - Exchange Segment
instrument required	enum string	Instrument type of the scrip . Refer Instrument
expiryCode optional	enum integer	Expiry of the instruments in case of derivatives. Refer Expiry Code
oi optional	boolean	Open Interest data for Futures & Options
fromDate required	string	Start date of the desired range
toDate required	string	End date of the desired range (non-inclusive)

Response Structure:

```
{
  "open": [...],
  "high": [...],
  "low": [...],
  "close": [...],
  "volume": [...],
  "timestamp": [...],
  "open_interest": [...]
}
```

Response Parameters:

Field	Type	Description
open	float	Open price of the timeframe
high	float	High price in the timeframe
low	float	Low price in the timeframe
close	float	Close price of the timeframe
volume	int	Volume traded in the timeframe
timestamp	int	Epoch timestamp
open_interest	int	Open Interest data for Futures & Options

Python Example:

```
dhan.historical_daily_data(security_id, exchange_segment, instrument_type, from_date, to_date)
```

Intraday Historical Data (`POST /charts/intraday`)

Request Structure (JSON):

```
{
  "securityId": "1333",
  "exchangeSegment": "NSE_EQ",
  "instrument": "EQUITY",
  "interval": "1",
  "oi": false,
  "fromDate": "2024-09-11 09:30:00",
  "toDate": "2024-09-15 13:00:00"
}
```

Parameters:

Field	Field Type	Description

securityId required	string	Exchange standard ID for each scrip. Refer Instrument List
exchangeSegment required	enum string	Exchange & segment for which data is to be fetched - Exchange Segment
instrument required	enum string	Instrument type of the scrip . Refer Instrument
interval required	enum integer	Minute intervals in timeframe `1` , `5` , `15` , `25` , `60`
oi optional	boolean	Open Interest data for Futures & Options
fromDate required	string	Start date of the desired range
toDate required	string	End date of the desired range

Note: The data size is very large in this scenario and only 90 days of data can be polled at once for any of the above time intervals. It is recommended that you store this data at your end for day-to-day analysis.

Response Structure:

```
{
  "open": [...],
  "high": [...],
  "low": [...],
  "close": [...],
  "volume": [...],
  "timestamp": [...],
  "open_interest": [...]
}
```

Python Example:

```
dhan.intraday_minute_data(security_id, exchange_segment, instrument_type, from_date, to_date)
```

Option Chain API

This API gives entire Option Chain of any Option Instrument, across exchanges and segments - for NSE, BSE and MCX traded options. With Option Chain, you get OI, greeks, volume, top bid/ask and price data of all strikes of a particular underlying.

Endpoints:
- `POST /optionchain` : Get Option Chain of any instrument
- `POST /optionchain/expirylist` : Expiry List for Options of Underlying

Info: You can call Option Chain API once every 3 seconds. This will give you latest updated data.

Option Chain (`POST /optionchain`)

Headers:

Header	Description
access-token required	Access Token generated via Dhan
client-id required	User specific identification generated by Dhan

Request Structure (JSON):

```
{
  "UnderlyingScrip":13,
  "UnderlyingSeg":"IDX_I",
  "Expiry":"2024-10-31"
}
```

Parameters:

Field	Field Type	Description
UnderlyingScrip required	int	Security ID of Underlying Instrument - can be found in Instrument List
UnderlyingSeg	enum string	Exchange & segment of underlying for which data is to be fetched - Exchange Segment
Expiry	string	Expiry Date of Option, for which Option Chain is requested. List of active expiries can be fetched from Expiry List

Response Structure:


```
{
  "data": {
    "last_price": 24964.25,
    "oc": {
      "25000.000000": {
        "ce": {
          "greeks": {
            "delta": 0.52546,
            "theta": -12.88756,
            "gamma": 0.00136,
            "vega": 12.98931
          },
          "implied_volatility": 8.945204889199001,
          "last_price": 125.05,
          "oi": 5962675,
          "previous_close_price": 190.45,
          "previous_oi": 3939375,
          "previous_volume": 831463,
```

```
            "top_ask_price": 124.9,
            "top_ask_quantity": 1000,
            "top_bid_price": 124,
            "top_bid_quantity": 100,
            "volume": 84202625
          },
          "pe": {
            "greeks": {
              "delta": -0.48099,
              "theta": -10.56587,
              "gamma": 0.00092,
              "vega": 13.00105
            },
            "implied_volatility": 13.321804909313869,
            "last_price": 165,
            "oi": 5059700,
            "previous_close_price": 153.6,
            "previous_oi": 4667700,
            "previous_volume": 1047989,
            "top_ask_price": 165,
            "top_ask_quantity": 375,
            "top_bid_price": 164.05,
            "top_bid_quantity": 50,
            "volume": 81097175
          }
        }
      }
    }
  }
```


Response Parameters:

Field	Type	Description
data.last_price	float	LTP of the Underlying
data.oc	array	Option Chain Array - Strike Wise
data.oc.{strike}	array	Strike Price for Underlying
data.oc.{strike}.ce	array	Call Option data of particular strike
data.oc.{strike}.pe	array	Put Option data of particular strike

Call/Put Option Data Parameters:

Field	Type	Description
greeks.delta	float	Measures the change of option\'s premium based on every 1 rupee change in underlying
greeks.theta	float	Measures how quickly an option\'s value decreases over time
greeks.gamma	float	Rate of change in an option\'s delta in relation to the price of the underlying asset

greeks.vega	float	Measures the change of option\'s premium in response to a 1% change in implied volatility
implied_volatility	float	Value of expected volatility of a stock over the life of the option
last_price	float	Last Traded Price of the Option Instrument
oi	int	Open Interest of the Option Instrument
previous_close_price	float	Previous day close price
previous_oi	int	Previous day Open Interest
previous_volume	int	Previous day volume
top_ask_price	float	Current best ask price available
top_ask_quantity	int	Quantity available at current best ask price
top_bid_price	float	Current best bid price available
top_bid_quantity	int	Quantity available at current best bid price
volume	int	Day volume for Option Instrument

Python Example:

```
dhan.option_chain(
  under_security_id=13,                # Nifty
  under_exchange_segment="IDX_I",
  expiry="2024-10-31"
)
```

Expiry List (`POST /optionchain/expirylist`)

Headers:
Header	Description
access-token required	Access Token generated via Dhan
client-id required	User specific identification generated by Dhan

Request Structure (JSON):

```
{
  "UnderlyingScrip":13,
  "UnderlyingSeg":"IDX_I"
}
```

Parameters:
Field	Field Type	Description
UnderlyingScrip required	int	Security ID of Underlying Instrument - can be found in

Instrument List |
| UnderlyingSeg | enum string | Exchange & segment of underlying for which data is to be fetched - Exchange Segment |

Response Structure:


```
{
  "data": [
    "2024-10-17",
    "2024-10-24",
    "2024-10-31",
    "2024-11-07",
    "2024-11-14",
    "2024-11-28",
    "2024-12-26",
    "2025-03-27",
    "2025-06-26",
    "2025-09-25",
    "2025-12-24",
    "2026-06-25",
    "2026-12-31",
    "2027-06-24",
    "2027-12-30",
    "2028-06-29",
    "2028-12-28",
    "2029-06-28"
  ],
  "status": "success"
}
```


Response Parameters:

Field	Type	Description
data[]	array	All expiry dates of underlying in `YYYY-MM-DD`

Python Example:

```
dhan.expiry_list(
  under_security_id=13,                # Nifty
  under_exchange_segment="IDX_I"
)
```

Instrument List API

You can fetch instrument list for all instruments which can be traded via Dhan by using the URLs below:

Compact: https://images.dhan.co/api-data/api-scrip-master.csv

Detailed: https://images.dhan.co/api-data/api-scrip-master-detailed.csv

These CSV files provide Security ID and other important details.

Segmentwise List

You can fetch detailed instrument list for all instruments in a particular exchange and segment by passing the same in parameters:

curl --location \'https://api.dhan.co/v2/instrument/{exchangeSegment}\' \

Python Example:

```
dhan.fetch_security_list("compact")
```

Column Description

Detailed tag	Compact tag	Description
EXCH_ID	SEM_EXM_EXCH_ID	Exchange NSE BSE MCX
SEGMENT	SEM_SEGMENT	Segment C - Currency D - Derivatives E - Equity M - Commodity
ISIN	-	International Securities Identification Number(ISIN) - 12-digit alphanumeric code unique for instruments
INSTRUMENT	SEM_INSTRUMENT_NAME	Instrument defined by

Detailed tag	Compact tag	Description
		Exchange - defined here
removed	SEM_EXPIRY_CODE	Expiry Code (applicable in case of Futures Contract) - defined Expiry Code
UNDERLYING_SECURITY_ID	-	Security ID of underlying instrument (applicable in case of derivative contracts)
UNDERLYING_SYMBOL	-	Symbol of underlying instrument (applicable in case of derivative contracts)
SYMBOL_NAME	SM_SYMBOL_NAME	Symbol name of instrument
removed	SEM_TRADING_SYMBOL	Exchange trading symbol of instrument
DISPLAY_NAME	SEM_CUSTOM_SYMBOL	Dhan display symbol name of instrument
INSTRUMENT_TYPE	SEM_EXCH_INSTRUMENT_TYPE	In addition to INSTRUMENT

Detailed tag	Compact tag	Description
		column, instrument type is defined by exchange adding more details about instrument
SERIES	SEM_SERIES	Exchange defined series for instrument
LOT_SIZE	SEM_LOT_UNITS	Lot Size in multiples of which instrument is traded
SM_EXPIRY_DATE	SEM_EXPIRY_DATE	Expiry date of instrument (applicable in case of derivative contracts)
STRIKE_PRICE	SEM_STRIKE_PRICE	Strike Price of Options Contract
OPTION_TYPE	SEM_OPTION_TYPE	Type of Options Contract `CE` - Call `PE` - Put
TICK_SIZE	SEM_TICK_SIZE	Minimum decimal point at which an instrument can be priced
EXPIRY_FLAG	SEM_EXPIRY_FLAG	

Detailed tag	Compact tag	Description
		Type of Expiry (applicable in case of option contracts) `M` - Monthly Expiry `W` - Weekly Expiry
BRACKET_FLAG	-	Bracket order status `N` - Not available `Y` - Allowed
COVER_FLAG	-	Cover order status `N` - Not available `Y` - Allowed
ASM_GSM_FLAG	-	Flag for instrument is ASM or GSM `N` - Not in ASM/GSM `R` - Removed from block `Y` - ASM/GSM
ASM_GSM_CATEGORY	-	Category of instrument in ASM or GSM `NA` in case of no surveillance
BUY_SELL_INDICATOR	-	Indicator to show if Buy and Sell is allowed in instrument `A`

Detailed tag	Compact tag	Description
		if both Buy/ Sell is allowed
BUY_CO_MIN_MARGIN_PER	-	Buy cover order minimum margin requirement (in percentage)
SELL_CO_MIN_MARGIN_PER	-	Sell cover order minimum margin requirement (in percentage)
BUY_CO_SL_RANGE_MAX_PERC	-	Buy cover order maximum range for stop loss leg (in percentage)
SELL_CO_SL_RANGE_MAX_PERC	-	Sell cover order maximum range for stop loss leg (in percentage)
BUY_CO_SL_RANGE_MIN_PERC	-	Buy cover order minimum range for stop loss leg (in percentage)

Detailed tag	Compact tag	Description
SELL_CO_SL_RANGE_MIN_PERC	-	Sell cover order minimum range for stop loss leg (in percentage)
BUY_BO_MIN_MARGIN_PER	-	Buy bracket order minimum margin requirement (in percentage)
SELL_BO_MIN_MARGIN_PER	-	Sell bracket order minimum margin requirement (in percentage)
BUY_BO_SL_RANGE_MAX_PERC	-	Buy bracket order maximum range for stop loss leg (in percentage)
SELL_BO_SL_RANGE_MAX_PERC	-	Sell bracket order maximum range for stop loss leg (in percentage)
BUY_BO_PROFIT_RANGE_MAX_PERC	-	Buy bracket order maximum

Detailed tag	Compact tag	Description
		range for target leg (in percentage)
SELL_BO_PROFIT_RANGE_MAX_PERC	-	Sell bracket order maximum range for target leg (in percentage)
BUY_BO_PROFIT_RANGE_MIN_PERC	-	Buy bracket order minimum range for target leg (in percentage)
SELL_BO_PROFIT_RANGE_MIN_PERC	-	Sell bracket order minimum range for target leg (in percentage)
MTF_LEVERAGE	-	MTF Leverage available (in x multiple) for eligible EQUITY instruments

Annexure

Exchange Segment

Attribute	Exchange	Segment	enum
IDX_I	Index	Index Value	0

Attribute	Exchange	Segment	enum
NSE_EQ	NSE	Equity Cash	1
NSE_FNO	NSE	Futures & Options	2
NSE_CURRENCY	NSE	Currency	3
BSE_EQ	BSE	Equity Cash	4
MCX_COMM	MCX	Commodity	5
BSE_CURRENCY	BSE	Currency	7
BSE_FNO	BSE	Futures & Options	8

Product Type

CO & BO product types will be valid only for Intraday.

Attribute	Detail
CNC	Cash & Carry for equity deliveries
INTRADAY	Intraday for Equity, Futures & Options
MARGIN	Carry Forward in Futures & Options
CO	Cover Order
BO	Bracket Order

Order Status

Attribute	Detail
TRANSIT	Did not reach the exchange server
PENDING	Awaiting execution
CLOSED	Used for Super Order, once both the entry and exit orders are placed
TRIGGERED	Used for Super Order, if Target or Stop Loss leg is triggered
REJECTED	Rejected by broker/exchange
CANCELLED	Cancelled by user
PART_TRADED	Partial Quantity traded successfully
TRADED	Executed successfully

After Market Order time

Attribute	Detail
PRE_OPEN	AMO pumped at pre-market session
OPEN	AMO pumped at market open
OPEN_30	AMO pumped 30 minutes after market open
OPEN_60	AMO pumped 60 minutes after market open

Expiry Code

Attribute	Detail
0	Current Expiry/Near Expiry
1	Next Expiry
2	Far Expiry

Instrument

Attribute	Detail
INDEX	Index
FUTIDX	Futures of Index
OPTIDX	Options of Index
EQUITY	Equity
FUTSTK	Futures of Stock
OPTSTK	Options of Stock
FUTCOM	Futures of Commodity
OPTFUT	Options of Commodity Futures
FUTCUR	Futures of Currency
OPTCUR	Options of Currency

Feed Request Code

Attribute	Detail
11	Connect Feed
12	Disconnect Feed
15	Subscribe - Ticker Packet
16	Unsubscribe - Ticker Packet
17	Subscribe - Quote Packet
18	Unsubscribe - Quote Packet
21	Subscribe - Full Packet
22	Unsubscribe - Full Packet
23	Subscribe - 20 Level Market Depth
24	Unsubscribe - 20 Level Market Depth

Feed Response Code

Attribute	Detail
1	Index Packet
2	Ticker Packet
4	Quote Packet
5	OI Packet
6	Prev Close Packet
7	Market Status Packet
8	Full Packet
50	Feed Disconnect

Trading API Error

Type	Code	Message
Invalid Authentication	DH-901	Client ID or user generated access token is invalid or expired.
Invalid Access	DH-902	User has not subscribed to Data APIs or does not have access to Trading APIs. Kindly subscribe to Data APIs to be able to fetch Data.
User Account	DH-903	Errors related to User\'s Account. Check if the required segments are activated or other requirements are met.
Rate Limit	DH-904	Too many requests on server from single user breaching rate limits. Try throttling API calls.
Input Exception	DH-905	Missing required fields, bad values for parameters etc.
Order Error	DH-906	Incorrect request for order and cannot be processed.
Data Error	DH-907	System is unable to fetch data due to incorrect parameters or no data present.
Internal Server Error	DH-908	Server was not able to process API request. This will only occur rarely.
Network Error	DH-909	Network error where the API was unable to communicate with the backend system.
Others	DH-910	Error originating from other reasons.

Data API Error

Code	Description
800	Internal Server Error
804	Requested number of instruments exceeds limit
805	Too many requests or connections. Further requests may result in the user being blocked.
806	Data APIs not subscribed

Code	Description
807	Access token is expired
808	Authentication Failed - Client ID or Access Token invalid
809	Access token is invalid
810	Client ID is invalid
811	Invalid Expiry Date
812	Invalid Date Format
813	Invalid SecurityId
814	Invalid Request

Live Order Update API

Real-time order updates of all your orders can be received directly via WebSocket in your system. The messages sent over this WebSocket will be JSON.

WebSocket Endpoint: `wss://api-order-update.dhan.co`

Establishing Connection

While establishing connection, you need to send an Authorisation Message for connection.

For Individual Users:

Authorisation message structure:

```
{
  "LoginReq":{
    "MsgCode": 42,
    "ClientId":"1000000001",
    "Token":"JWT"
  },
  "UserType": "SELF"
}
```

Parameters:
| Field | Type | Description |

| --- | --- | --- |
LoginReq required	object	JSON for adding Client ID and Access Token
MsgCode required	int	Message Code for getting Order Updates `42` by default
ClientId required	string	User specific identification generated by Dhan
Token required	string	Access Token generated for user
UserType required	string	`SELF` for individual users

For Partners:

Authorisation message structure:

```
{
  "LoginReq":{
    "MsgCode": 42,
    "ClientId": "partner_id"
  },
  "UserType": "PARTNER",
  "Secret": "partner_secret"
}
```

Parameters:

Field	Type	Description
LoginReq required	object	JSON for adding Client ID and Access Token
MsgCode required	int	Message Code for getting Order Updates `42` by default
ClientId required	string	`partner_id` generated for the partner
UserType required	string	`PARTNER` for partner platforms
Secret required	string	`partner_secret` generated for the partner

Order Update Message Structure:

```
{
  "Data": {
    "series": "EQ",
    "goodTillDaysDate": "2024-09-11",
    "instrumentType": "EQ",
    "refLtp": 13.21,
    "tickSize": 0.01,
    "algoId": "0",
    "multiplier": 1,
    "Exchange": "NSE",
    "Segment": "E",
    "Source": "N",
    "SecurityId": "14366",
    "ClientId": "1000000001",
```

```
        "ExchOrderNo": "1400000000404591",
        "OrderNo": "1124091136546",
        "Product": "C",
        "TxnType": "B",
        "OrderType": "LMT",
        "Validity": "DAY",
        "DiscQuantity": 1,
        "DiscQtyRem": 1,
        "RemainingQuantity": 1,
        "Quantity": 1,
        "TradedQty": 0,
        "Price": 13,
        "TriggerPrice": 0,
        "TradedPrice": 0,
        "AvgTradedPrice": 0,
        "AlgoOrdNo": null,
        "OffMktFlag": "0",
        "OrderDateTime": "2024-09-11 14:39:29",
        "ExchOrderTime": "2024-09-11 14:39:29",
        "LastUpdatedTime": "2024-09-11 14:39:29",
        "Remarks": "NR",
        "MktType": "NL",
        "ReasonDescription": "CONFIRMED",
        "LegNo": 1,
        "Instrument": "EQUITY",
        "Symbol": "IDEA",
        "ProductName": "CNC",
        "Status": "Cancelled",
        "LotSize": 1,
        "StrikePrice": null,
        "ExpiryDate": "0001-01-01 00:00:00",
        "OptType": "XX",
        "DisplayName": "Vodafone Idea",
        "Isin": "INE669E01016",
        "Series": "EQ",
        "GoodTillDaysDate": "2024-09-11",
        "RefLtp": 13.21,
        "TickSize": 0.01,
        "AlgoId": "0",
        "Multiplier": 1,
        "CorrelationId": "",
        "Remarks": "Super Order"
    },
    "Type": "order_alert"
}
```


Parameters:

Field	Type	Description
Exchange	string	Exchange in which order is placed

Segment	string	Segment for which order is placed
Source	string	Platform via which order is placed - `P` for API Orders
SecurityId	string	Exchange standard ID for each scrip. Refer Instrument List
ClientId	string	User specific identification generated by Dhan
ExchOrderNo	string	Order specific identification generated by Exchange
OrderNo	string	Order specific identification generated by Dhan
Product	enum string	Product type of trade `C` for CNC, `I` for INTRADAY, `M` for MARGIN, `F` for MTF , `V` for CO, `B` for BO
TxnType	enum string	The trading side of transaction `B` for Buy `S` for Sell
OrderType	enum string	Order Type `LMT` for Limit `MKT` for Market `SL` for Stop Loss `SLM` for Stop Loss
Validity	enum string	Validity of Order `DAY` `IOC`
DiscQuantity	int	Number of shares visible
DiscQtyRem	int	Disclosed quantity pending for execution
RemainingQuantity	int	Quantity pending for execution
Quantity	int	Total order quantity placed
TradedQty	int	Actual quantity executed on exchange
Price	float	Price at which order is placed
TriggerPrice	float	Price at which order is triggered, for SL-M, SL-L, CO & BO
TradedPrice	float	Price at which trade of an order is executed
AvgTradedPrice	float	Average trade price of an order (this will be different from `Traded Price` in case of partial execution)
AlgoOrdNo	float	Entry leg order number to track Target and Stop Loss leg (in case of `BO` and `CO`)
OffMktFlag	string	`1` in case of AMO order else `0`
OrderDateTime	string	Time at which the order is received by Dhan
ExchOrderTime	string	Time at which order is placed on Exchange
LastUpdatedTime	string	Last update time of any order modification or trade
Remarks	string	Additional remarks sent along while placing order
MktType	string	`NL` for Normal Market `AU` , `A1` and `A2` for Auction Market
ReasonDescription	string	Order rejection reason
LegNo	int	`1` for Entry Leg `2` for Stop Loss Leg `3` for Target Leg
Instrument	string	Instrument in which order is placed - Instrument List
Symbol	string	Symbol in which order is placed - Refer Instrument List
ProductName	string	Product type of the order placed - Product Type
Status	enum string	Last updated status of the order `TRANSIT` `PENDING` `REJECTED` `CANCELLED` `TRADED` `EXPIRED`
LotSize	int	Lot Size in case of Derivatives
StrikePrice	float	Strike Price in which order is placed in Option contract
ExpiryDate	string	Expiry Date of the contract in which order is placed

OptType	string	`CE` or `PE` in case of Option contract
DisplayName	string	Name of instrument in which order is placed - Refer Instrument List
Isin	string	ISIN of the instrument in which order is placed
Series	string	Exchange series of the instrument
GoodTillDaysDate	string	Order validity in case of Forever Order
RefLtp	float	LTP at time of order update
TickSize	float	Tick size of the instrument
AlgoId	string	Exchange ID for special order types
Multiplier	int	In case of commodity and currency contracts
CorrelationId	string	The user/partner generated id for tracking back
Remarks	string	`Super Order` if the order is part of super order

Python Example:

```
 # Live Order Update (WebSocket)
# This requires a separate WebSocket connection setup as detailed in the documentation.
```

Data APIs

Market Quote API

This API gives you snapshots of multiple instruments at once. You can fetch LTP, Quote or Market Depth of instruments via API which sends real time data at the time of API request.

Endpoints:
- `POST /marketfeed/ltp` : Get ticker data of instruments
- `POST /marketfeed/ohlc` : Get OHLC data of instruments
- `POST /marketfeed/quote` : Get market depth data of instruments

Info: You can fetch upto 1000 instruments in single API request with rate limit of 1 request per second.

Ticker Data (`POST /marketfeed/ltp`)

Headers:
Header	Description

| access-token required | Access Token generated via Dhan |
| client-id required | User specific identification generated by Dhan |

Request Structure (JSON):

```
{
  "NSE_EQ":[11536],
  "NSE_FNO":[49081,49082]
}
```

Parameters:
Field	Field Type	Description
Exchange Segment ENUM required	array	Security ID - can be found in Instrument List

Response Structure:

```
{
  "data": {
    "NSE_EQ": {
      "11536": {
        "last_price": 4520
      }
    },
    "NSE_FNO": {
      "49081": {
        "last_price": 368.15
      },
      "49082": {
        "last_price": 694.35
      }
    }
  },
  "status": "success"
}
```

Response Parameters:
Field	Type	Description
last_price	float	LTP of the Instrument

Python Example:

```
dhan.ohlc_data(
  securities = {"NSE_EQ":[1333]} # This function can be used for ticker data as well.
)
```

OHLC Data (`POST /marketfeed/ohlc`)

Headers:

Header	Description
access-token required	Access Token generated via Dhan
client-id required	User specific identification generated by Dhan

Request Structure (JSON):

```
{
  "NSE_EQ":[11536],
  "NSE_FNO":[49081,49082]
}
```

Parameters:

Field	Field Type	Description
Exchange Segment ENUM required	array	Security ID - can be found in Instrument List

Response Structure:

```
{
  "data": {
    "NSE_EQ": {
      "11536": {
        "last_price": 4525.55,
        "ohlc": {
          "open": 4521.45,
          "close": 4507.85,
          "high": 4530,
          "low": 4500
        }
      }
    },
    "NSE_FNO": {
      "49081": {
        "last_price": 368.15,
        "ohlc": {
          "open": 0,
```

```
            "close": 368.15,
            "high": 0,
            "low": 0
          }
        },
        "49082": {
          "last_price": 694.35,
          "ohlc": {
            "open": 0,
            "close": 694.35,
            "high": 0,
            "low": 0
          }
        }
      }
    },
    "status": "success"
}
```


Response Parameters:

Field	Type	Description
last_price	float	LTP of the Instrument
ohlc.open	float	Market opening price of the day
ohlc.close	float	Market closing price of the day
ohlc.high	float	Day High price
ohlc.low	float	Day Low price

Python Example:

```
dhan.ohlc_data(
  securities = {"NSE_EQ":[1333]}
)
```

Market Depth Data (`POST /marketfeed/quote`)

Headers:

Header	Description
access-token required	Access Token generated via Dhan
client-id required	User specific identification generated by Dhan

Request Structure (JSON):

```
{
  "NSE_FNO":[49081]
}
```

Parameters:

Field	Field Type	Description
Exchange Segment ENUM required	array	Security ID - can be found in Instrument List

Response Structure:


```
{
  "data": {
    "NSE_FNO": {
      "49081": {
        "average_price": 0,
        "buy_quantity": 1825,
        "depth": {
          "buy": [
            {
              "quantity": 1800,
              "orders": 1,
              "price": 77
            },
            {
              "quantity": 25,
              "orders": 1,
              "price": 50
            },
            {
              "quantity": 0,
              "orders": 0,
              "price": 0
            },
            {
              "quantity": 0,
              "orders": 0,
              "price": 0
            },
            {
              "quantity": 0,
              "orders": 0,
              "price": 0
            }
          ],
          "sell": [
            {
```

```
                    "quantity": 0,
                    "orders": 0,
                    "price": 0
                },
                {
                    "quantity": 0,
                    "orders": 0,
                    "price": 0
                },
                {
                    "quantity": 0,
                    "orders": 0,
                    "price": 0
                },
                {
                    "quantity": 0,
                    "orders": 0,
                    "price": 0
                },
                {
                    "quantity": 0,
                    "orders": 0,
                    "price": 0
                },
                {
                    "quantity": 0,
                    "orders": 0,
                    "price": 0
                }
            ]
        },
        "last_price": 368.15,
        "last_quantity": 0,
        "last_trade_time": "01/01/1980 00:00:00",
        "lower_circuit_limit": 48.25,
        "net_change": 0,
        "ohlc": {
            "open": 0,
            "close": 368.15,
            "high": 0,
            "low": 0
        },
        "oi": 0,
        "oi_day_high": 0,
        "oi_day_low": 0,
        "sell_quantity": 0,
        "upper_circuit_limit": 510.85,
        "volume": 0
      }
    }
  },
  "status": "success"
}
```


Response Parameters:

| Field | Type | Description |

| --- | --- | --- |
average_price	float	Volume weighted average price of the day
buy_quantity	int	Total buy order quantity pending at the exchange
sell_quantity	int	Total sell order quantity pending at the exchange
depth.buy.quantity	int	Number of quantity at this price depth
depth.buy.orders	int	Number of open BUY orders at this price depth
depth.buy.price	float	Price at which the BUY depth stands
depth.sell.quantity	int	Number of quantity at this price depth
depth.sell.orders	int	Number of open SELL orders at this price depth
depth.sell.price	float	Price at which the SELL depth stands
last_price	float	LTP of the Instrument
last_quantity	int	Last traded quantity
last_trade_time	string	Last trade time
lower_circuit_limit	float	Lower circuit limit
net_change	float	Net change in price
ohlc.open	float	Market opening price of the day
ohlc.close	float	Market closing price of the day
ohlc.high	float	Day High price
ohlc.low	float	Day Low price
oi	int	Open Interest
oi_day_high	int	Open Interest Day High
oi_day_low	int	Open Interest Day Low
volume	int	Total traded volume

Live Market Feed API

Real-time Market Data across exchanges and segments can now be availed on your system via WebSocket. WebSocket keeps a persistent connection open, allowing the server to push real-time data to your systems.

WebSocket Endpoint: `wss://api-feed.dhan.co?version=2&token=eyxxxxx&clientId=100xxxxxxx&authType=2`

Query Parameters for Connection:
Field	Description
version required	`2` for DhanHQ v2
token required	Access Token generated via Dhan
clientId required	User specific identification generated by Dhan
authType required	`2` by Default

Establishing Connection

While establishing connection, you need to send an Authorisation Message for connection.

For Individual Users:

Authorisation message structure:

```
{
  "LoginReq":{
    "MsgCode": 42,
    "ClientId":"1000000001",
    "Token":"JWT"
  },
  "UserType": "SELF"
}
```

Parameters:

Field	Type	Description
LoginReq required	object	JSON for adding Client ID and Access Token
MsgCode required	int	Message Code for getting Order Updates `42` by default
ClientId required	string	User specific identification generated by Dhan
Token required	string	Access Token generated for user
UserType required	string	`SELF` for individual users

For Partners:

Authorisation message structure:

```
{
  "LoginReq":{
    "MsgCode": 42,
    "ClientId": "partner_id"
  },
  "UserType": "PARTNER",
  "Secret": "partner_secret"
}
```

Parameters:

Field	Type	Description
LoginReq required	object	JSON for adding Client ID and Access Token
MsgCode required	int	Message Code for getting Order Updates `42` by default

ClientId required	string	`partner_id` generated for the partner
UserType required	string	`PARTNER` for partner platforms
Secret required	string	`partner_secret` generated for the partner

Adding Instruments

You can subscribe upto 5000 instruments in a single connection and receive market data packets. You can send multiple messages over a single connection to subscribe to all instruments and receive data (max 100 instruments per message).

Request Structure (JSON):

```
{
  "RequestCode" : 15,
  "InstrumentCount" : 2,
  "InstrumentList" : [
    {
      "ExchangeSegment" : "NSE_EQ",
      "SecurityId" : "1333"
    },
    {
      "ExchangeSegment" : "BSE_EQ",
      "SecurityId" : "532540"
    }
  ]
}
```

Parameters:
Field	Type	Description
RequestCode required	int	Code for subscribing to particular data mode. Refer to Feed Request Code to subscribe to required data mode
InstrumentCount required	int	No. of instruments to subscribe from this request
InstrumentList.ExchangeSegment required	enum string	Exchange Segment of instrument to be subscribed as found in Annexure
InstrumentList.SecurityId required	string	Exchange standard ID for each scrip. Refer Instrument List

Keeping Connection Alive

To keep the WebSocket connection alive, the server sends **Ping-Pong** messages every 10 seconds. If the client does not respond for more than 40 seconds, the connection is closed.

Market Data

The market feed data is sent as structured binary packets. All responses from Dhan Market Feed consists of Response Header and Payload.

Response Header (8 bytes):

Bytes	Type	Size	Description
1	[ ] byte	1	Feed Response Code can be referred in Annexure
2-3	int16	2	Message Length of the entire payload packet
4	[ ] byte	1	Exchange Segment can be referred in Annexure
5-8	int32	4	Security ID - can be found in Instrument List

Ticker Packet:
This packet consists of Last Traded Price (LTP) and Last Traded Time (LTT) data.

Bytes	Type	Size	Description
0-8	[ ] array	8	Response Header with code 2
9-12	float32	4	Last Traded Price of the subscribed instrument
13-16	int32	4	Last Trade Time

Prev Close Packet:
This packet provides Previous Day data for comparison.

Bytes	Type	Size	Description
0-8	[ ] array	8	Response Header with code 6
9-12	float32	4	Previous day closing price
13-16	int32	4	Open Interest - previous day

Quote Packet:
This packet contains complete trade data, LTP, and other information.

Bytes	Type	Size	Description
0-8	[ ] array	8	Response Header with code 4
9-12	float32	4	Latest Traded Price of the subscribed instrument
13-14	int16	2	Last Traded Quantity
15-18	int32	4	Last Trade Time (LTT)
19-22	float32	4	Average Trade Price (ATP)
23-26	int32	4	Volume
27-30	int32	4	Total Sell Quantity

31-34	int32	4	Total Buy Quantity
35-38	float32	4	Day Open Value
39-42	float32	4	Day Close Value - only sent post market close
43-46	float32	4	Day High Value
47-50	float32	4	Day Low Value

OI Data Packet:
Open Interest (OI) data for Derivative Contracts.

Bytes	Type	Size	Description
0-8	[ ] array	8	Response Header with code 5
9-12	int32	4	Open Interest of the contract

Full Packet:
This packet contains complete trade data, Market Depth, and OI data in a single packet.

Bytes	Type	Size	Description
0-8	[ ] array	8	Response Header with code 8
9-12	float32	4	Latest Traded Price of the subscribed instrument
13-14	int16	2	Last Traded Quantity
15-18	int32	4	Last Trade Time (LTT)
19-22	float32	4	Average Trade Price (ATP)
23-26	int32	4	Volume
27-30	int32	4	Total Sell Quantity
31-34	int32	4	Total Buy Quantity
35-38	int32	4	Open Interest in the contract (for Derivatives)
39-42	int32	4	Highest Open Interest for the day (only for NSE_FNO)
43-46	int32	4	Lowest Open Interest for the day (only for NSE_FNO)
47-50	float32	4	Day Open Value
51-54	float32	4	Day Close Value - only sent post market close
55-58	float32	4	Day High Value
59-62	float32	4	Day Low Value
63-162	Market Depth Structure	100	5 packets of 20 bytes each for each instrument

Python Example:

```
 # Live Market Feed (WebSocket)
# This requires a separate WebSocket connection setup as detailed in the
documentation.
```

20 Market Depth API

Level 3 data includes market depth upto 20 levels, streamed real-time via websockets. This data can be used to detect demand supply zones.

Note: Only NSE Equity and Derivatives segments are enabled for 20 Level Market Depth.

WebSocket Endpoint: `wss://depth-api-feed.dhan.co/twentydepth?token=eyxxxxx&clientId=100xxxxxxx&authType=2`

Query Parameters for Connection:

Field	Description
token required	Access Token generated via Dhan
clientId required	User specific identification generated by Dhan
authType required	`2` by Default

Establishing Connection

(Same as Live Market Feed API)

Adding Instruments

You can subscribe upto 50 instruments in a single connection and receive market data packets. You can send all 50 instruments in a single JSON message or multiple messages.

Request Structure (JSON):

```
{
  "RequestCode" : 23,
  "InstrumentCount" : 2,
  "InstrumentList" : [
    {
      "ExchangeSegment" : "NSE_EQ",
      "SecurityId" : "1333"
    },
    {
      "ExchangeSegment" : "NSE_FNO",
      "SecurityId" : "532540"
    }
```

```
    ]
}
```

Parameters:
Field	Type	Description
RequestCode required	int	Code for subscribing to particular data mode. `23` for 20 Level Market Depth.
InstrumentCount required	int	No. of instruments to subscribe from this request
InstrumentList.ExchangeSegment required	enum string	Exchange Segment of instrument to be subscribed as found in Annexure
InstrumentList.SecurityId required	string	Exchange standard ID for each scrip. Refer Instrument List

Keeping Connection Alive

To keep the WebSocket connection alive, the server sends **Ping-Pong** messages every 10 seconds. If the client does not respond for more than 40 seconds, the connection is closed.

20-Level Depth Packet

The market depth data is sent as structured binary packet. All responses from Dhan Market Feed consists of Response Header and Payload.

Response Header (12 bytes):
Bytes	Type	Size	Description
`1-2`	int16	`2`	Message Length of the entire payload packet
`3`	[ ] byte	`1`	Feed Response Code can be referred in Annexure
`4`	[ ] byte	`1`	Exchange Segment can be referred in Annexure
`5-8`	int32	`4`	Security ID - can be found in Instrument List
`9-12`	uint32	`4`	Message Sequence (to be ignored)

Depth Packet:
Depth Data Packet for 20 level market depth is structured differently from 5 level depth. You will receive the bid (sell) and ask (buy) data packets separately, each containing 20 packets of 16 bytes each.

Bytes	Type	Size	Description
`0-12`	[ ] array	`12`	Response Header `41` for Bid Data (Buy) `51` for Ask Data (Sell)

| `13-332` | Bid/Ask Depth Structure | `320` | 20 packets of 16 bytes each for each instrument |

Each of these 20 packets will be received in the following packet structure:

Bytes	Type	Size	Description
`1-8`	float64	`8`	Price
`9-12`	uint32	`4`	Quantity
`13-16`	uint32	`4`	No. of Orders

Feed Disconnect:
To disconnect WebSocket, you can send the following JSON request message:

```
{
  "RequestCode" : 12
}
```

In case of WebSocket disconnection from server side, you will receive a disconnection packet with a reason code.

Note: If more than 5 websockets are established, the first socket will be disconnected with `805` with every additional connection.

Disconnection Packet Structure:
Bytes	Type	Size	Description
`0-12`	[ ] array	`8`	Response Header with code `50`
`13-14`	int16	`2`	Disconnection message code

Historical Data API

This API gives you historical candle data for the desired scrip across segments & exchange. This data is presented in the form of a candle and gives you timestamp, open, high, low, close & volume.

Endpoints:
- `POST /charts/historical` : Get OHLC for daily timeframe
- `POST /charts/intraday` : Get OHLC for minute timeframe

Daily Historical Data (`POST /charts/historical`)

Request Structure (JSON):

```
{
  "securityId": "1333",
  "exchangeSegment":"NSE_EQ",
  "instrument": "EQUITY",
  "expiryCode": 0,
  "oi": false,
  "fromDate": "2022-01-08",
  "toDate": "2022-02-08"
}
```

Parameters:

Field	Field Type	Description
securityId required	string	Exchange standard ID for each scrip. Refer Instrument List
exchangeSegment required	enum string	Exchange & segment for which data is to be fetched - Exchange Segment
instrument required	enum string	Instrument type of the scrip . Refer Instrument
expiryCode optional	enum integer	Expiry of the instruments in case of derivatives. Refer Expiry Code
oi optional	boolean	Open Interest data for Futures & Options
fromDate required	string	Start date of the desired range
toDate required	string	End date of the desired range (non-inclusive)

Response Structure:

```
{
  "open": [...],
  "high": [...],
  "low": [...],
  "close": [...],
  "volume": [...],
  "timestamp": [...],
  "open_interest": [...]
}
```

Response Parameters:

Field	Type	Description
open	float	Open price of the timeframe
high	float	High price in the timeframe
low	float	Low price in the timeframe
close	float	Close price of the timeframe
volume	int	Volume traded in the timeframe

| timestamp | int | Epoch timestamp |
| open_interest | int | Open Interest data for Futures & Options |

Python Example:

```
 dhan.historical_daily_data(security_id, exchange_segment, instrument_type,
from_date, to_date)
```

Intraday Historical Data (`POST /charts/intraday`)

Request Structure (JSON):

```
{
  "securityId": "1333",
  "exchangeSegment": "NSE_EQ",
  "instrument": "EQUITY",
  "interval": "1",
  "oi": false,
  "fromDate": "2024-09-11 09:30:00",
  "toDate": "2024-09-15 13:00:00"
}
```

Parameters:
Field	Field Type	Description
securityId required	string	Exchange standard ID for each scrip. Refer Instrument List
exchangeSegment required	enum string	Exchange & segment for which data is to be fetched - Exchange Segment
instrument required	enum string	Instrument type of the scrip . Refer Instrument
interval required	enum integer	Minute intervals in timeframe `1` , `5` , `15` , `25` , `60`
oi optional	boolean	Open Interest data for Futures & Options
fromDate required	string	Start date of the desired range
toDate required	string	End date of the desired range

Note: The data size is very large in this scenario and only 90 days of data can be polled at once for any of the above time intervals. It is recommended that you store this data at your end for day-to-day analysis.

Response Structure:

```
{
  "open": [...],
  "high": [...],
  "low": [...],
```

```
    "close": [...],
    "volume": [...],
    "timestamp": [...],
    "open_interest": [...]
}
```

Python Example:

```
dhan.intraday_minute_data(security_id, exchange_segment, instrument_type,
from_date, to_date)
```

Option Chain API

This API gives entire Option Chain of any Option Instrument, across exchanges and segments - for NSE, BSE and MCX traded options. With Option Chain, you get OI, greeks, volume, top bid/ask and price data of all strikes of a particular underlying.

Endpoints:
- `POST /optionchain` : Get Option Chain of any instrument
- `POST /optionchain/expirylist` : Expiry List for Options of Underlying

Info: You can call Option Chain API once every 3 seconds. This will give you latest updated data.

Option Chain (`POST /optionchain`)

Headers:
Header	Description
access-token required	Access Token generated via Dhan
client-id required	User specific identification generated by Dhan

Request Structure (JSON):

```
{
  "UnderlyingScrip":13,
  "UnderlyingSeg":"IDX_I",
  "Expiry":"2024-10-31"
}
```

Parameters:
Field	Field Type	Description

UnderlyingScrip required	int	Security ID of Underlying Instrument - can be found in Instrument List
UnderlyingSeg	enum string	Exchange & segment of underlying for which data is to be fetched - Exchange Segment
Expiry	string	Expiry Date of Option, for which Option Chain is requested. List of active expiries can be fetched from Expiry List

Response Structure:


```
{
  "data": {
    "last_price": 24964.25,
    "oc": {
      "25000.000000": {
        "ce": {
          "greeks": {
            "delta": 0.52546,
            "theta": -12.88756,
            "gamma": 0.00136,
            "vega": 12.98931
          },
          "implied_volatility": 8.945204889199001,
          "last_price": 125.05,
          "oi": 5962675,
          "previous_close_price": 190.45,
          "previous_oi": 3939375,
          "previous_volume": 831463,
          "top_ask_price": 124.9,
          "top_ask_quantity": 1000,
          "top_bid_price": 124,
          "top_bid_quantity": 100,
          "volume": 84202625
        },
        "pe": {
          "greeks": {
            "delta": -0.48099,
            "theta": -10.56587,
            "gamma": 0.00092,
            "vega": 13.00105
          },
          "implied_volatility": 13.321804909313869,
          "last_price": 165,
          "oi": 5059700,
          "previous_close_price": 153.6,
          "previous_oi": 4667700,
          "previous_volume": 1047989,
          "top_ask_price": 165,
          "top_ask_quantity": 375,
          "top_bid_price": 164.05,
```

```
            "top_bid_quantity": 50,
            "volume": 81097175
          }
        }
      }
    }
  }
```


Response Parameters:

Field	Type	Description
data.last_price	float	LTP of the Underlying
data.oc	array	Option Chain Array - Strike Wise
data.oc.{strike}	array	Strike Price for Underlying
data.oc.{strike}.ce	array	Call Option data of particular strike
data.oc.{strike}.pe	array	Put Option data of particular strike

Call/Put Option Data Parameters:

Field	Type	Description
greeks.delta	float	Measures the change of option\"s premium based on every 1 rupee change in underlying
greeks.theta	float	Measures how quickly an option\"s value decreases over time
greeks.gamma	float	Rate of change in an option\"s delta in relation to the price of the underlying asset
greeks.vega	float	Measures the change of option\"s premium in response to a 1% change in implied volatility
implied_volatility	float	Value of expected volatility of a stock over the life of the option
last_price	float	Last Traded Price of the Option Instrument
oi	int	Open Interest of the Option Instrument
previous_close_price	float	Previous day close price
previous_oi	int	Previous day Open Interest
previous_volume	int	Previous day volume
top_ask_price	float	Current best ask price available
top_ask_quantity	int	Quantity available at current best ask price
top_bid_price	float	Current best bid price available
top_bid_quantity	int	Quantity available at current best bid price
volume	int	Day volume for Option Instrument

Python Example:

```
dhan.option_chain(
  under_security_id=13,                 # Nifty
  under_exchange_segment="IDX_I",
  expiry="2024-10-31"
)
```

Expiry List (`POST /optionchain/expirylist`)

Headers:

Header	Description
access-token required	Access Token generated via Dhan
client-id required	User specific identification generated by Dhan

Request Structure (JSON):

```
{
  "UnderlyingScrip":13,
  "UnderlyingSeg":"IDX_I"
}
```

Parameters:

Field	Field Type	Description
UnderlyingScrip required	int	Security ID of Underlying Instrument - can be found in Instrument List
UnderlyingSeg	enum string	Exchange & segment of underlying for which data is to be fetched - Exchange Segment

Response Structure:

```
{
  "data": [
    "2024-10-17",
    "2024-10-24",
    "2024-10-31",
    "2024-11-07",
    "2024-11-14",
    "2024-11-28",
    "2024-12-26",
    "2025-03-27",
    "2025-06-26",
    "2025-09-25",
    "2025-12-24",
    "2026-06-25",
```

```
        "2026-12-31",
        "2027-06-24",
        "2027-12-30",
        "2028-06-29",
        "2028-12-28",
        "2029-06-28"
    ],
    "status": "success"
}
```

Response Parameters:

Field	Type	Description
data[]	array	All expiry dates of underlying in `YYYY-MM-DD`

Python Example:

```
dhan.expiry_list(
    under_security_id=13,                # Nifty
    under_exchange_segment="IDX_I"
)
```

Instrument List API

You can fetch instrument list for all instruments which can be traded via Dhan by using the URLs below:

Compact: `https://images.dhan.co/api-data/api-scrip-master.csv`

Detailed: `https://images.dhan.co/api-data/api-scrip-master-detailed.csv`

These CSV files provide Security ID and other important details.

Segmentwise List

You can fetch detailed instrument list for all instruments in a particular exchange and segment by passing the same in parameters:

`curl --location \'https://api.dhan.co/v2/instrument/{exchangeSegment}\' \`

Python Example:

```
dhan.fetch_security_list("compact")
```

Column Description

Detailed tag	Compact tag	Description
EXCH_ID	SEM_EXM_EXCH_ID	Exchange NSE BSE MCX
SEGMENT	SEM_SEGMENT	Segment C - Currency D - Derivatives E - Equity M - Commodity
ISIN	-	International Securities Identification Number(ISIN) - 12-digit alphanumeric code unique for instruments
INSTRUMENT	SEM_INSTRUMENT_NAME	Instrument defined by Exchange - defined here
removed	SEM_EXPIRY_CODE	Expiry Code (applicable in case of Futures Contract) - defined Expiry Code
UNDERLYING_SECURITY_ID	-	Security ID of underlying instrument (applicable in case of

Detailed tag	Compact tag	Description
		derivative contracts)
UNDERLYING_SYMBOL	-	Symbol of underlying instrument (applicable in case of derivative contracts)
SYMBOL_NAME	SM_SYMBOL_NAME	Symbol name of instrument
removed	SEM_TRADING_SYMBOL	Exchange trading symbol of instrument
DISPLAY_NAME	SEM_CUSTOM_SYMBOL	Dhan display symbol name of instrument
INSTRUMENT_TYPE	SEM_EXCH_INSTRUMENT_TYPE	In addition to INSTRUMENT column, instrument type is defined by exchange adding more details about instrument
SERIES	SEM_SERIES	Exchange defined series for instrument
LOT_SIZE	SEM_LOT_UNITS	Lot Size in multiples of which

Detailed tag	Compact tag	Description
		instrument is traded
SM_EXPIRY_DATE	SEM_EXPIRY_DATE	Expiry date of instrument (applicable in case of derivative contracts)
STRIKE_PRICE	SEM_STRIKE_PRICE	Strike Price of Options Contract
OPTION_TYPE	SEM_OPTION_TYPE	Type of Options Contract CE - Call PE - Put
TICK_SIZE	SEM_TICK_SIZE	Minimum decimal point at which an instrument can be priced
EXPIRY_FLAG	SEM_EXPIRY_FLAG	Type of Expiry (applicable in case of option contracts) M - Monthly Expiry W - Weekly Expiry
BRACKET_FLAG	-	Bracket order status N - Not available Y - Allowed
COVER_FLAG	-	Cover order status N - Not

Detailed tag	Compact tag	Description
		available `Y` - Allowed
`ASM_GSM_FLAG`	-	Flag for instrument is ASM or GSM `N` - Not in ASM/GSM `R` - Removed from block `Y` - ASM/GSM
`ASM_GSM_CATEGORY`	-	Category of instrument in ASM or GSM `NA` in case of no surveillance
`BUY_SELL_INDICATOR`	-	Indicator to show if Buy and Sell is allowed in instrument `A` if both Buy/ Sell is allowed
`BUY_CO_MIN_MARGIN_PER`	-	Buy cover order minimum margin requirement (in percentage)
`SELL_CO_MIN_MARGIN_PER`	-	Sell cover order minimum margin requirement

Detailed tag	Compact tag	Description
		(in percentage)
BUY_CO_SL_RANGE_MAX_PERC	-	Buy cover order maximum range for stop loss leg (in percentage)
SELL_CO_SL_RANGE_MAX_PERC	-	Sell cover order maximum range for stop loss leg (in percentage)
BUY_CO_SL_RANGE_MIN_PERC	-	Buy cover order minimum range for stop loss leg (in percentage)
SELL_CO_SL_RANGE_MIN_PERC	-	Sell cover order minimum range for stop loss leg (in percentage)
BUY_BO_MIN_MARGIN_PER	-	Buy bracket order minimum margin requirement (in percentage)
SELL_BO_MIN_MARGIN_PER	-	Sell bracket order

Detailed tag	Compact tag	Description
		minimum margin requirement (in percentage)
BUY_BO_SL_RANGE_MAX_PERC	-	Buy bracket order maximum range for stop loss leg (in percentage)
SELL_BO_SL_RANGE_MAX_PERC	-	Sell bracket order maximum range for stop loss leg (in percentage)
BUY_BO_PROFIT_RANGE_MAX_PERC	-	Buy bracket order maximum range for target leg (in percentage)
SELL_BO_PROFIT_RANGE_MAX_PERC	-	Sell bracket order maximum range for target leg (in percentage)
BUY_BO_PROFIT_RANGE_MIN_PERC	-	Buy bracket order minimum range for target leg (in percentage)

Detailed tag	Compact tag	Description
SELL_BO_PROFIT_RANGE_MIN_PERC	-	Sell bracket order minimum range for target leg (in percentage)
MTF_LEVERAGE	-	MTF Leverage available (in x multiple) for eligible EQUITY instruments

Annexure

Exchange Segment

Attribute	Exchange	Segment	enum
IDX_I	Index	Index Value	0
NSE_EQ	NSE	Equity Cash	1
NSE_FNO	NSE	Futures & Options	2
NSE_CURRENCY	NSE	Currency	3
BSE_EQ	BSE	Equity Cash	4
MCX_COMM	MCX	Commodity	5
BSE_CURRENCY	BSE	Currency	7
BSE_FNO	BSE	Futures & Options	8

Product Type

CO & BO product types will be valid only for Intraday.

Attribute	Detail
CNC	Cash & Carry for equity deliveries
INTRADAY	Intraday for Equity, Futures & Options
MARGIN	Carry Forward in Futures & Options
CO	Cover Order
BO	Bracket Order

Order Status

Attribute	Detail
TRANSIT	Did not reach the exchange server
PENDING	Awaiting execution
CLOSED	Used for Super Order, once both the entry and exit orders are placed
TRIGGERED	Used for Super Order, if Target or Stop Loss leg is triggered
REJECTED	Rejected by broker/exchange
CANCELLED	Cancelled by user
PART_TRADED	Partial Quantity traded successfully
TRADED	Executed successfully

After Market Order time

Attribute	Detail
PRE_OPEN	AMO pumped at pre-market session
OPEN	AMO pumped at market open
OPEN_30	AMO pumped 30 minutes after market open
OPEN_60	AMO pumped 60 minutes after market open

Expiry Code

Attribute	Detail
0	Current Expiry/Near Expiry
1	Next Expiry
2	Far Expiry

Instrument

Attribute	Detail
INDEX	Index
FUTIDX	Futures of Index
OPTIDX	Options of Index
EQUITY	Equity
FUTSTK	Futures of Stock
OPTSTK	Options of Stock
FUTCOM	Futures of Commodity
OPTFUT	Options of Commodity Futures
FUTCUR	Futures of Currency
OPTCUR	Options of Currency

Feed Request Code

Attribute	Detail
`11`	Connect Feed
`12`	Disconnect Feed
`15`	Subscribe - Ticker Packet
`16`	Unsubscribe - Ticker Packet
`17`	Subscribe - Quote Packet
`18`	Unsubscribe - Quote Packet
`21`	Subscribe - Full Packet
`22`	Unsubscribe - Full Packet
`24`	Unsubscribe - 20 Level Market Depth

Feed Response Code

Attribute	Detail
1	Index Packet
2	Ticker Packet
4	Quote Packet
5	OI Packet
6	Prev Close Packet
7	Market Status Packet
8	Full Packet
50	Feed Disconnect

Trading API Error

Type	Code	Message
Invalid Authentication	DH-901	Client ID or user generated access token is invalid or expired.
Invalid Access	DH-902	User has not subscribed to Data APIs or does not have access to Trading APIs. Kindly subscribe to Data APIs to be able to fetch Data.
User Account	DH-903	Errors related to User\"s Account. Check if the required segments are activated or other requirements are met.
Rate Limit	DH-904	Too many requests on server from single user breaching rate limits. Try throttling API calls.
Input Exception	DH-905	Missing required fields, bad values for parameters etc.
Order Error	DH-906	Incorrect request for order and cannot be processed.
Data Error	DH-907	System is unable to fetch data due to incorrect parameters or no data present.
	DH-908	

Type	Code	Message
Internal Server Error		Server was not able to process API request. This will only occur rarely.
Network Error	DH-909	Network error where the API was unable to communicate with the backend system.
Others	DH-910	Error originating from other reasons.

Data API Error

Code	Description
800	Internal Server Error
804	Requested number of instruments exceeds limit
805	Too many requests or connections. Further requests may result in the user being blocked.
806	Data APIs not subscribed
807	Access token is expired
808	Authentication Failed - Client ID or Access Token invalid
809	Access token is invalid
810	Client ID is invalid
811	Invalid Expiry Date
812	Invalid Date Format
813	Invalid SecurityId
814	Invalid Request

Annexure

Exchange Segment

Attribute	Exchange	Segment	enum
IDX_I	Index	Index Value	0
NSE_EQ	NSE	Equity Cash	1
NSE_FNO	NSE	Futures & Options	2
NSE_CURRENCY	NSE	Currency	3
BSE_EQ	BSE	Equity Cash	4
MCX_COMM	MCX	Commodity	5
BSE_CURRENCY	BSE	Currency	7
BSE_FNO	BSE	Futures & Options	8

Product Type

CO & BO product types will be valid only for Intraday.

Attribute	Detail
CNC	Cash & Carry for equity deliveries
INTRADAY	Intraday for Equity, Futures & Options
MARGIN	Carry Forward in Futures & Options
CO	Cover Order
BO	Bracket Order

Order Status

Attribute	Detail
TRANSIT	Did not reach the exchange server
PENDING	Awaiting execution

CLOSED	Used for Super Order, once both the entry and exit orders are placed
TRIGGERED	Used for Super Order, if Target or Stop Loss leg is triggered
REJECTED	Rejected by broker/exchange
CANCELLED	Cancelled by user
PART_TRADED	Partial Quantity traded successfully
TRADED	Executed successfully

After Market Order time

Attribute	Detail
PRE_OPEN	AMO pumped at pre-market session
OPEN	AMO pumped at market open
OPEN_30	AMO pumped 30 minutes after market open
OPEN_60	AMO pumped 60 minutes after market open

Expiry Code

Attribute	Detail
0	Current Expiry/Near Expiry
1	Next Expiry
2	Far Expiry

Instrument

Attribute	Detail
INDEX	Index
FUTIDX	Futures of Index
OPTIDX	Options of Index
EQUITY	Equity
FUTSTK	Futures of Stock
OPTSTK	Options of Stock
FUTCOM	Futures of Commodity

Attribute	Detail
OPTFUT	Options of Commodity Futures
FUTCUR	Futures of Currency
OPTCUR	Options of Currency

Feed Request Code

Attribute	Detail
11	Connect Feed
12	Disconnect Feed
15	Subscribe - Ticker Packet
16	Unsubscribe - Ticker Packet
17	Subscribe - Quote Packet
18	Unsubscribe - Quote Packet
21	Subscribe - Full Packet
22	Unsubscribe - Full Packet
23	Subscribe - 20 Level Market Depth
24	Unsubscribe - 20 Level Market Depth

Feed Response Code

Attribute	Detail
1	Index Packet
2	Ticker Packet
4	Quote Packet
5	OI Packet
6	Prev Close Packet

Attribute	Detail
7	Market Status Packet
8	Full Packet
50	Feed Disconnect

Trading API Error

Type	Code	Message
Invalid Authentication	DH-901	Client ID or user generated access token is invalid or expired.
Invalid Access	DH-902	User has not subscribed to Data APIs or does not have access to Trading APIs. Kindly subscribe to Data APIs to be able to fetch Data.
User Account	DH-903	Errors related to User\'s Account. Check if the required segments are activated or other requirements are met.
Rate Limit	DH-904	Too many requests on server from single user breaching rate limits. Try throttling API calls.
Input Exception	DH-905	Missing required fields, bad values for parameters etc.
Order Error	DH-906	Incorrect request for order and cannot be processed.
Data Error	DH-907	System is unable to fetch data due to incorrect parameters or no data present.
Internal Server Error	DH-908	Server was not able to process API request. This will only occur rarely.
Network Error	DH-909	Network error where the API was unable to communicate with the backend system.
Others	DH-910	Error originating from other reasons.

Data API Error

Code	Description
800	Internal Server Error
804	Requested number of instruments exceeds limit
805	Too many requests or connections. Further requests may result in the user being blocked.
806	Data APIs not subscribed
807	Access token is expired
808	Authentication Failed - Client ID or Access Token invalid
809	Access token is invalid
810	Client ID is invalid
811	Invalid Expiry Date
812	Invalid Date Format
813	Invalid SecurityId
814	Invalid Request

Portfolio API

This API lets you retrieve holdings and positions in your portfolio.

Endpoints:
- `GET /holdings` : Retrieve list of holdings in demat account
- `GET /positions` : Retrieve open positions
- `POST /positions/convert` : Convert intraday position to delivery or delivery to intraday

Holdings (`GET /holdings`)

Request Structure: No Body

Response Structure:

```
[
  {
```

```
    "exchange": "ALL",
    "tradingSymbol": "HDFC",
    "securityId": "1330",
    "isin": "INE001A01036",
    "totalQty": 1000,
    "dpQty": 1000,
    "t1Qty": 0,
    "availableQty": 1000,
    "collateralQty": 0,
    "avgCostPrice": 2655.0
  }
]
```


Parameters:

Field	Type	Description
exchange	enum string	Exchange
tradingSymbol	string	Refer Trading Symbol at Page No
securityId	string	Exchange standard ID for each scrip. Refer Instrument List
isin	string	Universal standard ID for each scrip
totalQty	int	Total quantity
dpQty	int	Quantity delivered in demat account
t1Qty	int	Quantity pending delivered in demat account
availableQty	int	Quantity available for transaction
collateralQty	int	Quantity placed as collateral with broker
avgCostPrice	float	Average Buy Price of total quantity

Python Example:

```
dhan.get_holdings()
```

Positions (`GET /positions`)

Request Structure: No Body

Response Structure:


```
[
  {
    "dhanClientId": "1000000009",
    "tradingSymbol": "TCS",
    "securityId": "11536",
    "positionType": "LONG",
    "exchangeSegment": "NSE_EQ",
    "productType": "CNC",
```

```
        "buyAvg": 3345.8,
        "buyQty": 40,
        "costPrice": 3215.0,
        "sellAvg": 0.0,
        "sellQty": 0,
        "netQty": 40,
        "realizedProfit": 0.0,
        "unrealizedProfit": 6122.0,
        "rbiReferenceRate": 1.0,
        "multiplier": 1,
        "carryForwardBuyQty": 0,
        "carryForwardSellQty": 0,
        "carryForwardBuyValue": 0.0,
        "carryForwardSellValue": 0.0,
        "dayBuyQty": 40,
        "daySellQty": 0,
        "dayBuyValue": 133832.0,
        "daySellValue": 0.0,
        "drvExpiryDate": "0001-01-01",
        "drvOptionType": null,
        "drvStrikePrice": 0.0,
        "crossCurrency": false
    }
]
```


Parameters:

Field	Type	Description
dhanClientId	string	User specific identification generated by Dhan
tradingSymbol	string	Refer Trading Symbol in Tables
securityId	string	Exchange standard id for each scrip. Refer Instrument List
positionType	enum string	Position Type `LONG` `SHORT` `CLOSED`
exchangeSegment	enum string	Exchange & Segment `NSE_EQ` `NSE_FNO` `NSE_CURRENCY` `BSE_EQ` `BSE_FNO` `BSE_CURRENCY` `MCX_COMM`
productType	enum string	Product type `CNC` `INTRADAY` `MARGIN` `MTF` `CO` `BO`
buyAvg	float	Average buy price mark to market
buyQty	int	Total quantity bought
costPrice	int	Actual Cost Price
sellAvg	float	Average sell price mark to market
sellQty	int	Total quantities sold
netQty	int	buyQty - sellQty = netQty
realizedProfit	float	Profit or loss booked
unrealizedProfit	float	Profit or loss standing for open position
rbiReferenceRate	float	RBI mandated reference rate for forex
multiplier	int	Multiplier

carryForwardBuyQty	int	Carry forward buy quantity
carryForwardSellQty	int	Carry forward sell quantity
carryForwardBuyValue	float	Carry forward buy value
carryForwardSellValue	float	Carry forward sell value
dayBuyQty	int	Day buy quantity
daySellQty	int	Day sell quantity
dayBuyValue	float	Day buy value
daySellValue	float	Day sell value
drvExpiryDate	string	Derivative expiry date
drvOptionType	string	Derivative option type
drvStrikePrice	float	Derivative strike price
crossCurrency	boolean	Cross currency flag

Python Example:

```
dhan.get_positions()
```

Convert Position (`POST /positions/convert`)

Python Example: (No direct example in GitHub, but the API exists)

EDIS API

To sell holding stocks, one needs to complete the CDSL eDIS flow, generate T-PIN & mark stock to complete the sell action.

Python Examples:

```
 # Generate TPIN
dhan.generate_tpin()

# Enter TPIN in Form
dhan.open_browser_for_tpin(
    isin=\'INE00IN01015\',
    qty=1,
    exchange=\'NSE\'
)

# EDIS Status and Inquiry
dhan.edis_inquiry()
```

Trader\'s Control API (Kill Switch)

This API lets you activate or deactivate the kill switch for your account, which will disable trading for the current trading day.

Endpoint:
- `POST /killswitch` : Activate/Deactivate Kill Switch

Request Structure (URL Parameters):
`https://api.dhan.co/v2/killswitch?killSwitchStatus=ACTIVATE` (or `DEACTIVATE`)

Request Headers:
- `Accept: application/json`
- `Content-Type: application/json`
- `access-token: JWT`

Request Body: No Body

Response Structure:

```
{
  "dhanClientId":"1000000009",
  "killSwitchStatus": "Kill Switch has been successfully activated"
}
```

Response Parameters:
Field	Type	Description
dhanClientId	string	User specific identification generated by Dhan
killSwitchStatus	string	Status of Kill Switch - activated or not

Note: You need to ensure that all your positions are closed and there are no pending orders in your account to be able to activate Kill Switch.

Python Example:

```
dhan.kill_switch(
  kill_switch_status=dhan.ACTIVATE
)
```

Funds API

Users can get details about the fund requirements or available funds (with margin requirements) in their Trading Account.

Endpoints:

- `POST /margincalculator` : Margin requirement for any order
- `GET /fundlimit` : Retrieve trading account fund information

Margin Calculator (`POST /margincalculator`)

Request Structure (JSON):

```
{
  "dhanClientId": "1000000132",
  "exchangeSegment": "NSE_EQ",
  "transactionType": "BUY",
  "quantity": 5,
  "productType": "CNC",
  "securityId": "1333",
  "price": 1428
}
```

Parameters:

Field	Type	Description
dhanClientId	string	User specific identification generated by Dhan
exchangeSegment	string	Exchange Segment of instrument to be subscribed as found in Annexure
transactionType	string	The trading side of transaction `BUY` `SELL`
quantity	int	Number of shares for the order
productType	string	Product type `CNC` `INTRADAY` `MARGIN` `MTF` `CO` `BO`
securityId	string	Exchange standard ID for each scrip. Refer Instrument List
price	float	Price at which order is placed

Response Structure:

```
{
  "dhanClientId": "1000000132",
  "marginRequired": 1000.00,
  "adhocMargin": 0.00,
  "spanMargin": 0.00,
  "exposureMargin": 0.00,
  "optionPremium": 0.00,
  "varMargin": 0.00,
  "premiumMargin": 0.00,
  "brokerage": 0.00,
  "totalTax": 0.00,
  "totalCharges": 0.00,
```

```
    "leverage": 0.00
}
```

Response Parameters:

Field	Type	Description
dhanClientId	string	User specific identification generated by Dhan
marginRequired	float	Total margin required for the order
adhocMargin	float	Adhoc margin
spanMargin	float	SPAN margin
exposureMargin	float	Exposure margin
optionPremium	float	Option premium
varMargin	float	VAR margin
premiumMargin	float	Premium margin
brokerage	float	Brokerage
totalTax	float	Total tax
totalCharges	float	Total charges
leverage	float	Leverage

Fund Limit (`GET /fundlimit`)

Request Structure: No Body

Response Structure:

```
{
    "dhanClientId": "1000000132",
    "availableBalance": 100000.00,
    "withdrawableBalance": 90000.00,
    "cashMargin": 80000.00,
    "collateralMargin": 10000.00,
    "intradayPayin": 5000.00,
    "approvedPayin": 5000.00,
    "pendingPayin": 0.00,
    "utilizedMargin": 10000.00,
    "m2mUnrealized": 500.00,
    "m2mRealized": 200.00,
    "holdingSales": 1000.00,
    "adhocMargin": 0.00,
    "spanMargin": 0.00,
    "exposureMargin": 0.00,
    "optionPremium": 0.00,
    "varMargin": 0.00,
    "premiumMargin": 0.00,
    "brokerage": 0.00,
    "totalTax": 0.00,
```

```
    "totalCharges": 0.00,
    "leverage": 0.00
}
```

Response Parameters:

Field	Type	Description
dhanClientId	string	User specific identification generated by Dhan
availableBalance	float	Available balance for trading
withdrawableBalance	float	Withdrawable balance
cashMargin	float	Cash margin
collateralMargin	float	Collateral margin
intradayPayin	float	Intraday payin
approvedPayin	float	Approved payin
pendingPayin	float	Pending payin
utilizedMargin	float	Utilized margin
m2mUnrealized	float	Mark to market unrealized profit/loss
m2mRealized	float	Mark to market realized profit/loss
holdingSales	float	Holding sales
adhocMargin	float	Adhoc margin
spanMargin	float	SPAN margin
exposureMargin	float	Exposure margin
optionPremium	float	Option premium
varMargin	float	VAR margin
premiumMargin	float	Premium margin
brokerage	float	Brokerage
totalTax	float	Total tax
totalCharges	float	Total charges
leverage	float	Leverage

Python Example:

```
dhan.get_fund_limits()
```

Statement API

This set of APIs retrieves all your trade and ledger details to help you summarize and analyze your trades.

Endpoints:
- `GET /ledger` : Retrieve Trading Account debit and credit details
- `GET /trades/` : Retrieve historical trade data

Ledger Report (`GET /ledger`)

Query Parameters:
Field	Description
from-date required	Date from which Ledger Report is required in format `YYYY-MM-DD`
to-date required	Date upto which Ledger Report is required in format `YYYY-MM-DD`

Request Structure: No Body

Response Structure:


```
{
  "dhanClientId": "1000000001",
  "narration": "FUNDS WITHDRAWAL",
  "voucherdate": "Jun 22, 2022",
  "exchange": "NSE-CAPITAL",
  "voucherdesc": "PAYBNK",
  "vouchernumber": "202200036701",
  "debit": "20000.00",
  "credit": "0.00",
  "runbal": "957.29"
}
```


Parameters:
Field	Type	Description
dhanClientId	string	User specific identification generated by Dhan
narration	string	Description of the ledger transaction
voucherdate	string	Transaction Date
exchange	string	Exchange information for the transaction
voucherdesc	string	Nature of transaction
vouchernumber	string	System generated transaction number
debit	string	Debit amount (only when credit returns 0)
credit	string	Credit amount (only when debit returns 0)
runbal	string	Running Balance post transaction

Trade History (`GET /trades/{from-date}/{to-date}/{page}`)

Path Parameters:

Field	Description
from-date required	Date from which Trade History is required in format `YYYY-MM-DD`
to-date required	Date upto which Trade History is required in format `YYYY-MM-DD`
page required	Page number of which data is being fetched. Pass `0` as default.

Request Structure: No Body

Response Structure:


```
[
  {
    "dhanClientId": "1000000001",
    "orderId": "212212307731",
    "exchangeOrderId": "76036896",
    "exchangeTradeId": "407958",
    "transactionType": "BUY",
    "exchangeSegment": "NSE_EQ",
    "productType": "CNC",
    "orderType": "MARKET",
    "tradingSymbol": null,
    "customSymbol": "Tata Motors",
    "securityId": "3456",
    "tradedQuantity": 1,
    "tradedPrice": 390.9,
    "isin": "INE155A01022",
    "instrument": "EQUITY",
    "sebiTax": 0.0004,
    "stt": 0,
    "brokerageCharges": 0,
    "serviceTax": 0.0025,
    "exchangeTransactionCharges": 0.0135,
    "stampDuty": 0,
    "createTime": "NA",
    "updateTime": "NA",
    "exchangeTime": "2022-12-30 10:00:46",
    "drvExpiryDate": "NA",
    "drvOptionType": "NA",
    "drvStrikePrice": 0
  }
]
```


Parameters:

| Field | Type | Description |

| --- | --- | --- |
dhanClientId	string	User specific identification generated by Dhan
orderId	string	Order specific identification generated by Dhan
exchangeOrderId	string	Order specific identification generated by exchange
exchangeTradeId	string	Trade specific identification generated by exchange
transactionType	enum string	The trading side of transaction `BUY` `SELL`
exchangeSegment	enum string	Exchange Segment of instrument to be subscribed as found in Annexure
productType	enum string	Product type of trade `CNC` `INTRADAY` `MARGIN` `MTF` `CO` `BO`
orderType	enum string	Order Type `LIMIT` `MARKET` `STOP_LOSS` `STOP_LOSS_MARKET`
tradingSymbol	string	Refer Trading Symbol in Tables
customSymbol	string	Trading Symbol as per Dhan
securityId	string	Exchange standard ID for each scrip. Refer Instrument List
tradedQuantity	int	Number of shares executed
tradedPrice	float	Price at which trade is executed
isin	string	Universal standard ID for each scrip
instrument	string	Type of Instrument `EQUITY` `DERIVATIVES`
sebiTax	string	SEBI Turnover Charges
stt	string	Securities Transactions Tax
brokerageCharges	string	Brokerage charges by Dhan, refer pricing here
serviceTax	string	Applicable Service Tax
exchangeTransactionCharges	string	Exchange Transaction Charge
stampDuty	string	Stamp Duty Charges
createTime	string	Time at which the order is created
updateTime	string	Time at which the last activity happened
exchangeTime	string	Time at which order reached at exchange
drvExpiryDate	int	For F&O, expiry date of contract
drvOptionType	enum string	Type of Option `CALL` `PUT`
drvStrikePrice	float	For Options, Strike Price

Python Example:

```
dhan.get_trade_history(from_date,to_date,page_number=0)
```

Postback API

Postback API or Webhooks uses a `POST` request with **JSON payload** to the Postback URL. This JSON payload contains order updates in case of status changes or modifications.

Postback Payload Structure:


```
{
  "dhanClientId": "1000000003",
  "orderId": "112111182198",
  "correlationId":"123abc678",
  "orderStatus": "PENDING",
  "transactionType": "BUY",
  "exchangeSegment": "NSE_EQ",
  "productType": "INTRADAY",
  "orderType": "MARKET",
  "validity": "DAY",
  "tradingSymbol": "",
  "securityId": "11536",
  "quantity": 5,
  "disclosedQuantity": 0,
  "price": 0.0,
  "triggerPrice": 0.0,
  "afterMarketOrder": false,
  "boProfitValue": 0.0,
  "boStopLossValue": 0.0,
  "legName": null,
  "createTime": "2021-11-24 13:33:03",
  "updateTime": "2021-11-24 13:33:03",
  "exchangeTime": "2021-11-24 13:33:03",
  "drvExpiryDate": null,
  "drvOptionType": null,
  "drvStrikePrice": 0.0,
  "omsErrorCode": null,
  "omsErrorDescription": null
}
```


Parameters:

Field	Type	Description
dhanClientId	string	User specific identification generated by Dhan
orderId	string	Order specific identification generated by Dhan
correlationId	string	The user/partner generated id for tracking back
orderStatus	enum string	Last updated status of the order `TRANSIT` `PENDING` `REJECTED` `CANCELLED` `TRADED` `EXPIRED`
transactionType	enum string	The trading side of transaction `BUY` `SELL`

exchangeSegment	enum string	Exchange & Segment `NSE_EQ` `NSE_FNO` `NSE_CURRENCY` `BSE_EQ` `MCX_COMM`
productType	enum string	Product type of trade `CNC` `INTRADAY` `MARGIN` `MTF` `CO` `BO`
orderType	enum string	Order Type `LIMIT` `MARKET` `STOP_LOSS` `STOP_LOSS_MARKET`
validity	enum string	Validity of Order `DAY` `IOC`
tradingSymbol	string	Refer Trading Symbol in Tables
securityId	string	Exchange standard id for each scrip. Refer Instrument List
quantity	int	Number of shares for the order
disclosedQuantity	int	Number of shares visible
price	float	Price at which order is placed
triggerPrice	float	Price at which order is triggered, for SL-M, SL-L, CO & BO
afterMarketOrder	boolean	The order placed is AMO ?
boProfitValue	float	Bracket Order Target Price change
boStopLossValue	float	Bracket Order Stop Loss Price change
legName	enum string	Leg identification in case of BO `ENTRY_LEG` `TARGET_LEG` `STOP_LOSS_LEG`
createTime	string	Time at which the order is created
updateTime	string	Time at which the last activity happened
exchangeTime	string	Time at which order reached at exchange
drvExpiryDate	int	For F&O, expiry date of contract
drvOptionType	enum string	Type of Option `CALL` `PUT`
drvStrikePrice	float	For Options, Strike Price
omserroeCode	string	Error code in case the order is rejected or failed
omsErrorDescription	string	Description of error in case the order is rejected or failed

Setting up Postback:
To set up Postback API, you will need to provide a unique Postback URL to receive callbacks. This is done by entering your URL in the \'Postback URL\' field while generating the access token on web.dhan.co.

Important: You will not receive postback calls if Postback URL is set to `localhost` or `127.0.0.1` .

Data APIs

Market Quote API

This API gives you snapshots of multiple instruments at once. You can fetch LTP, Quote or Market Depth of instruments via API which sends real time data at the time of API request.

Endpoints:
- `POST /marketfeed/ltp` : Get ticker data of instruments
- `POST /marketfeed/ohlc` : Get OHLC data of instruments
- `POST /marketfeed/quote` : Get market depth data of instruments

Info: You can fetch upto 1000 instruments in single API request with rate limit of 1 request per second.

Ticker Data (`POST /marketfeed/ltp`)

Headers:
Header	Description
access-token required	Access Token generated via Dhan
client-id required	User specific identification generated by Dhan

Request Structure (JSON):

```
{
  "NSE_EQ":[11536],
  "NSE_FNO":[49081,49082]
}
```

Parameters:
Field	Field Type	Description
Exchange Segment ENUM required	array	Security ID - can be found in Instrument List

Response Structure:

```
{
  "data": {
    "NSE_EQ": {
      "11536": {
```

```
            "last_price": 4520
          }
        },
        "NSE_FNO": {
          "49081": {
            "last_price": 368.15
          },
          "49082": {
            "last_price": 694.35
          }
        }
    },
    "status": "success"
}
```

Response Parameters:

Field	Type	Description
last_price	float	LTP of the Instrument

Python Example:

```
dhan.ohlc_data(
  securities = {"NSE_EQ":[1333]} # This function can be used for ticker data as well.
)
```

OHLC Data (`POST /marketfeed/ohlc`)

Headers:

Header	Description
access-token required	Access Token generated via Dhan
client-id required	User specific identification generated by Dhan

Request Structure (JSON):

```
{
  "NSE_EQ":[11536],
  "NSE_FNO":[49081,49082]
}
```

Parameters:

Field	Field Type	Description

| Exchange Segment ENUM required | array | Security ID - can be found in Instrument List |

Response Structure:


```
{
  "data": {
    "NSE_EQ": {
      "11536": {
        "last_price": 4525.55,
        "ohlc": {
          "open": 4521.45,
          "close": 4507.85,
          "high": 4530,
          "low": 4500
        }
      }
    },
    "NSE_FNO": {
      "49081": {
        "last_price": 368.15,
        "ohlc": {
          "open": 0,
          "close": 368.15,
          "high": 0,
          "low": 0
        }
      },
      "49082": {
        "last_price": 694.35,
        "ohlc": {
          "open": 0,
          "close": 694.35,
          "high": 0,
          "low": 0
        }
      }
    }
  },
  "status": "success"
}
```


Response Parameters:

Field	Type	Description
last_price	float	LTP of the Instrument
ohlc.open	float	Market opening price of the day
ohlc.close	float	Market closing price of the day

| ohlc.high | float | Day High price |
| ohlc.low | float | Day Low price |

Python Example:

```
dhan.ohlc_data(
  securities = {"NSE_EQ":[1333]}
)
```

Market Depth Data (`POST /marketfeed/quote`)

Headers:
Header	Description
access-token required	Access Token generated via Dhan
client-id required	User specific identification generated by Dhan

Request Structure (JSON):

```
{
  "NSE_FNO":[49081]
}
```

Parameters:
Field	Field Type	Description
Exchange Segment ENUM required	array	Security ID - can be found in Instrument List

Response Structure:

```
{
  "data": {
    "NSE_FNO": {
      "49081": {
        "average_price": 0,
        "buy_quantity": 1825,
        "depth": {
          "buy": [
            {
              "quantity": 1800,
              "orders": 1,
              "price": 77
            },
            {
```

```
            "quantity": 25,
            "orders": 1,
            "price": 50
          },
          {
            "quantity": 0,
            "orders": 0,
            "price": 0
          },
          {
            "quantity": 0,
            "orders": 0,
            "price": 0
          },
          {
            "quantity": 0,
            "orders": 0,
            "price": 0
          }
        ],
        "sell": [
          {
            "quantity": 0,
            "orders": 0,
            "price": 0
          },
          {
            "quantity": 0,
            "orders": 0,
            "price": 0
          },
          {
            "quantity": 0,
            "orders": 0,
            "price": 0
          },
          {
            "quantity": 0,
            "orders": 0,
            "price": 0
          },
          {
            "quantity": 0,
            "orders": 0,
            "price": 0
          },
          {
            "quantity": 0,
            "orders": 0,
            "price": 0
          }
        ]
      },
      "last_price": 368.15,
      "last_quantity": 0,
      "last_trade_time": "01/01/1980 00:00:00",
      "lower_circuit_limit": 48.25,
```

```
            "net_change": 0,
            "ohlc": {
              "open": 0,
              "close": 368.15,
              "high": 0,
              "low": 0
            },
            "oi": 0,
            "oi_day_high": 0,
            "oi_day_low": 0,
            "sell_quantity": 0,
            "upper_circuit_limit": 510.85,
            "volume": 0
          }
        }
    },
    "status": "success"
}
```


Response Parameters:

Field	Type	Description
average_price	float	Volume weighted average price of the day
buy_quantity	int	Total buy order quantity pending at the exchange
sell_quantity	int	Total sell order quantity pending at the exchange
depth.buy.quantity	int	Number of quantity at this price depth
depth.buy.orders	int	Number of open BUY orders at this price depth
depth.buy.price	float	Price at which the BUY depth stands
depth.sell.quantity	int	Number of quantity at this price depth
depth.sell.orders	int	Number of open SELL orders at this price depth
depth.sell.price	float	Price at which the SELL depth stands
last_price	float	LTP of the Instrument
last_quantity	int	Last traded quantity
last_trade_time	string	Last trade time
lower_circuit_limit	float	Lower circuit limit
net_change	float	Net change in price
ohlc.open	float	Market opening price of the day
ohlc.close	float	Market closing price of the day
ohlc.high	float	Day High price
ohlc.low	float	Day Low price
oi	int	Open Interest
oi_day_high	int	Open Interest Day High
oi_day_low	int	Open Interest Day Low
volume	int	Total traded volume

Live Market Feed API

Real-time Market Data across exchanges and segments can now be availed on your system via WebSocket. WebSocket keeps a persistent connection open, allowing the server to push real-time data to your systems.

WebSocket Endpoint: `wss://api-feed.dhan.co?version=2&token=eyxxxxx&clientId=100xxxxxxx&authType=2`

Query Parameters for Connection:

Field	Description
version required	`2` for DhanHQ v2
token required	Access Token generated via Dhan
clientId required	User specific identification generated by Dhan
authType required	`2` by Default

Establishing Connection

While establishing connection, you need to send an Authorisation Message for connection.

For Individual Users:

Authorisation message structure:

```
{
  "LoginReq":{
    "MsgCode": 42,
    "ClientId":"1000000001",
    "Token":"JWT"
  },
  "UserType": "SELF"
}
```

Parameters:

Field	Type	Description
LoginReq required	object	JSON for adding Client ID and Access Token
MsgCode required	int	Message Code for getting Order Updates `42` by default
ClientId required	string	User specific identification generated by Dhan
Token required	string	Access Token generated for user
UserType required	string	`SELF` for individual users

For Partners:

Authorisation message structure:

```
{
  "LoginReq":{
    "MsgCode": 42,
    "ClientId": "partner_id"
  },
  "UserType": "PARTNER",
  "Secret": "partner_secret"
}
```

Parameters:

Field	Type	Description
LoginReq required	object	JSON for adding Client ID and Access Token
MsgCode required	int	Message Code for getting Order Updates `42` by default
ClientId required	string	`partner_id` generated for the partner
UserType required	string	`PARTNER` for partner platforms
Secret required	string	`partner_secret` generated for the partner

Adding Instruments

You can subscribe upto 5000 instruments in a single connection and receive market data packets. You can send multiple messages over a single connection to subscribe to all instruments and receive data (max 100 instruments per message).

Request Structure (JSON):

```
{
  "RequestCode" : 15,
  "InstrumentCount" : 2,
  "InstrumentList" : [
    {
      "ExchangeSegment" : "NSE_EQ",
      "SecurityId" : "1333"
    },
    {
      "ExchangeSegment" : "BSE_EQ",
      "SecurityId" : "532540"
    }
  ]
}
```

Parameters:
Field	Type	Description
RequestCode required	int	Code for subscribing to particular data mode. Refer to Feed Request Code to subscribe to required data mode
InstrumentCount required	int	No. of instruments to subscribe from this request
InstrumentList.ExchangeSegment required	enum string	Exchange Segment of instrument to be subscribed as found in Annexure
InstrumentList.SecurityId required	string	Exchange standard ID for each scrip. Refer Instrument List

Keeping Connection Alive

To keep the WebSocket connection alive, the server sends **Ping-Pong** messages every 10 seconds. If the client does not respond for more than 40 seconds, the connection is closed.

Market Data

The market feed data is sent as structured binary packets. All responses from Dhan Market Feed consists of Response Header and Payload.

Response Header (8 bytes):
Bytes	Type	Size	Description
1	[ ] byte	1	Feed Response Code can be referred in Annexure
2-3	int16	2	Message Length of the entire payload packet
4	[ ] byte	1	Exchange Segment can be referred in Annexure
5-8	int32	4	Security ID - can be found in Instrument List

Ticker Packet:
This packet consists of Last Traded Price (LTP) and Last Traded Time (LTT) data.

Bytes	Type	Size	Description
0-8	[ ] array	8	Response Header with code 2
9-12	float32	4	Last Traded Price of the subscribed instrument
13-16	int32	4	Last Trade Time

Prev Close Packet:
This packet provides Previous Day data for comparison.

Bytes	Type	Size	Description
0-8	[ ] array	8	Response Header with code 6
9-12	float32	4	Previous day closing price
13-16	int32	4	Open Interest - previous day

Quote Packet:
This packet contains complete trade data, LTP, and other information.

Bytes	Type	Size	Description
0-8	[ ] array	8	Response Header with code 4
9-12	float32	4	Latest Traded Price of the subscribed instrument
13-14	int16	2	Last Traded Quantity
15-18	int32	4	Last Trade Time (LTT)
19-22	float32	4	Average Trade Price (ATP)
23-26	int32	4	Volume
27-30	int32	4	Total Sell Quantity
31-34	int32	4	Total Buy Quantity
35-38	float32	4	Day Open Value
39-42	float32	4	Day Close Value - only sent post market close
43-46	float32	4	Day High Value
47-50	float32	4	Day Low Value

OI Data Packet:
Open Interest (OI) data for Derivative Contracts.

Bytes	Type	Size	Description
0-8	[ ] array	8	Response Header with code 5
9-12	int32	4	Open Interest of the contract

Full Packet:
This packet contains complete trade data, Market Depth, and OI data in a single packet.

Bytes	Type	Size	Description
0-8	[ ] array	8	Response Header with code 8
9-12	float32	4	Latest Traded Price of the subscribed instrument
13-14	int16	2	Last Traded Quantity
15-18	int32	4	Last Trade Time (LTT)
19-22	float32	4	Average Trade Price (ATP)

23-26	int32	4	Volume
27-30	int32	4	Total Sell Quantity
31-34	int32	4	Total Buy Quantity
35-38	int32	4	Open Interest in the contract (for Derivatives)
39-42	int32	4	Highest Open Interest for the day (only for NSE_FNO)
43-46	int32	4	Lowest Open Interest for the day (only for NSE_FNO)
47-50	float32	4	Day Open Value
51-54	float32	4	Day Close Value - only sent post market close
55-58	float32	4	Day High Value
59-62	float32	4	Day Low Value
63-162	Market Depth Structure	100	5 packets of 20 bytes each for each instrument

Python Example:

```
 # Live Market Feed (WebSocket)
# This requires a separate WebSocket connection setup as detailed in the documentation.
```

20 Market Depth API

Level 3 data includes market depth upto 20 levels, streamed real-time via websockets. This data can be used to detect demand supply zones.

Note: Only NSE Equity and Derivatives segments are enabled for 20 Level Market Depth.

WebSocket Endpoint: `wss://depth-api-feed.dhan.co/twentydepth?token=eyxxxxx&clientId=100xxxxxxx&authType=2`

Query Parameters for Connection:

Field	Description
token required	Access Token generated via Dhan
clientId required	User specific identification generated by Dhan
authType required	2 by Default

Establishing Connection

(Same as Live Market Feed API)

Adding Instruments

You can subscribe upto 50 instruments in a single connection and receive market data packets. You can send all 50 instruments in a single JSON message or multiple messages.

Request Structure (JSON):

```
{
  "RequestCode" : 23,
  "InstrumentCount" : 2,
  "InstrumentList" : [
    {
      "ExchangeSegment" : "NSE_EQ",
      "SecurityId" : "1333"
    },
    {
      "ExchangeSegment" : "NSE_FNO",
      "SecurityId" : "532540"
    }
  ]
}
```

Parameters:

Field	Type	Description
RequestCode required	int	Code for subscribing to particular data mode. `23` for 20 Level Market Depth.
InstrumentCount required	int	No. of instruments to subscribe from this request
InstrumentList.ExchangeSegment required	enum string	Exchange Segment of instrument to be subscribed as found in Annexure
InstrumentList.SecurityId required	string	Exchange standard ID for each scrip. Refer Instrument List

Keeping Connection Alive

To keep the WebSocket connection alive, the server sends **Ping-Pong** messages every 10 seconds. If the client does not respond for more than 40 seconds, the connection is closed.

20-Level Depth Packet

The market depth data is sent as structured binary packet. All responses from Dhan Market Feed consists of Response Header and Payload.

Response Header (12 bytes):

Bytes	Type	Size	Description
1-2	int16	2	Message Length of the entire payload packet
3	[ ] byte	1	Feed Response Code can be referred in Annexure
4	[ ] byte	1	Exchange Segment can be referred in Annexure
5-8	int32	4	Security ID - can be found in Instrument List
9-12	uint32	4	Message Sequence (to be ignored)

Depth Packet:

Depth Data Packet for 20 level market depth is structured differently from 5 level depth. You will receive the bid (sell) and ask (buy) data packets separately, each containing 20 packets of 16 bytes each.

Bytes	Type	Size	Description
0-12	[ ] array	12	Response Header 41 for Bid Data (Buy) 51 for Ask Data (Sell)
13-332	Bid/Ask Depth Structure	320	20 packets of 16 bytes each for each instrument

Each of these 20 packets will be received in the following packet structure:

Bytes	Type	Size	Description
1-8	float64	8	Price
9-12	uint32	4	Quantity
13-16	uint32	4	No. of Orders

Feed Disconnect:

To disconnect WebSocket, you can send the following JSON request message:

```
{
  "RequestCode" : 12
}
```

In case of WebSocket disconnection from server side, you will receive a disconnection packet with a reason code.

Note: If more than 5 websockets are established, the first socket will be disconnected with 805 with every additional connection.

Disconnection Packet Structure:

| Bytes | Type | Size | Description |

| --- | --- | --- |
| `0-12` | [ ] array | `8` | Response Header with code `50` |
| `13-14` | int16 | `2` | Disconnection message code |

Historical Data API

This API gives you historical candle data for the desired scrip across segments & exchange. This data is presented in the form of a candle and gives you timestamp, open, high, low, close & volume.

Endpoints:
- `POST /charts/historical` : Get OHLC for daily timeframe
- `POST /charts/intraday` : Get OHLC for minute timeframe

Daily Historical Data (`POST /charts/historical`)

Request Structure (JSON):

```
{
  "securityId": "1333",
  "exchangeSegment":"NSE_EQ",
  "instrument": "EQUITY",
  "expiryCode": 0,
  "oi": false,
  "fromDate": "2022-01-08",
  "toDate": "2022-02-08"
}
```

Parameters:
Field	Field Type	Description
securityId required	string	Exchange standard ID for each scrip. Refer Instrument List
exchangeSegment required	enum string	Exchange & segment for which data is to be fetched - Exchange Segment
instrument required	enum string	Instrument type of the scrip . Refer Instrument
expiryCode optional	enum integer	Expiry of the instruments in case of derivatives. Refer Expiry Code
oi optional	boolean	Open Interest data for Futures & Options
fromDate required	string	Start date of the desired range
toDate required	string	End date of the desired range (non-inclusive)

Response Structure:

```
{
  "open": [...],
  "high": [...],
  "low": [...],
  "close": [...],
  "volume": [...],
  "timestamp": [...],
  "open_interest": [...]
}
```

Response Parameters:

Field	Type	Description
open	float	Open price of the timeframe
high	float	High price in the timeframe
low	float	Low price in the timeframe
close	float	Close price of the timeframe
volume	int	Volume traded in the timeframe
timestamp	int	Epoch timestamp
open_interest	int	Open Interest data for Futures & Options

Python Example:

```
dhan.historical_daily_data(security_id, exchange_segment, instrument_type, from_date, to_date)
```

Intraday Historical Data (`POST /charts/intraday`)

Request Structure (JSON):

```
{
  "securityId": "1333",
  "exchangeSegment": "NSE_EQ",
  "instrument": "EQUITY",
  "interval": "1",
  "oi": false,
  "fromDate": "2024-09-11 09:30:00",
  "toDate": "2024-09-15 13:00:00"
}
```

Parameters:

Field	Field Type	Description

securityId required	string	Exchange standard ID for each scrip. Refer Instrument List
exchangeSegment required	enum string	Exchange & segment for which data is to be fetched - Exchange Segment
instrument required	enum string	Instrument type of the scrip . Refer Instrument
interval required	enum integer	Minute intervals in timeframe `1` , `5` , `15` , `25` , `60`
oi optional	boolean	Open Interest data for Futures & Options
fromDate required	string	Start date of the desired range
toDate required	string	End date of the desired range

Note: The data size is very large in this scenario and only 90 days of data can be polled at once for any of the above time intervals. It is recommended that you store this data at your end for day-to-day analysis.

Response Structure:

```
{
  "open": [...],
  "high": [...],
  "low": [...],
  "close": [...],
  "volume": [...],
  "timestamp": [...],
  "open_interest": [...]
}
```

Python Example:

```
dhan.intraday_minute_data(security_id, exchange_segment, instrument_type, from_date, to_date)
```

Option Chain API

This API gives entire Option Chain of any Option Instrument, across exchanges and segments - for NSE, BSE and MCX traded options. With Option Chain, you get OI, greeks, volume, top bid/ask and price data of all strikes of a particular underlying.

Endpoints:
- `POST /optionchain` : Get Option Chain of any instrument
- `POST /optionchain/expirylist` : Expiry List for Options of Underlying

Info: You can call Option Chain API once every 3 seconds. This will give you latest updated data.

Option Chain (`POST /optionchain`)

Headers:

Header	Description
access-token required	Access Token generated via Dhan
client-id required	User specific identification generated by Dhan

Request Structure (JSON):

```
{
  "UnderlyingScrip":13,
  "UnderlyingSeg":"IDX_I",
  "Expiry":"2024-10-31"
}
```

Parameters:

Field	Field Type	Description
UnderlyingScrip required	int	Security ID of Underlying Instrument - can be found in Instrument List
UnderlyingSeg	enum string	Exchange & segment of underlying for which data is to be fetched - Exchange Segment
Expiry	string	Expiry Date of Option, for which Option Chain is requested. List of active expiries can be fetched from Expiry List

Response Structure:


```
{
  "data": {
    "last_price": 24964.25,
    "oc": {
      "25000.000000": {
        "ce": {
          "greeks": {
            "delta": 0.52546,
            "theta": -12.88756,
            "gamma": 0.00136,
            "vega": 12.98931
          },
          "implied_volatility": 8.945204889199001,
          "last_price": 125.05,
          "oi": 5962675,
          "previous_close_price": 190.45,
          "previous_oi": 3939375,
          "previous_volume": 831463,
```

```
            "top_ask_price": 124.9,
            "top_ask_quantity": 1000,
            "top_bid_price": 124,
            "top_bid_quantity": 100,
            "volume": 84202625
          },
          "pe": {
            "greeks": {
              "delta": -0.48099,
              "theta": -10.56587,
              "gamma": 0.00092,
              "vega": 13.00105
            },
            "implied_volatility": 13.321804909313869,
            "last_price": 165,
            "oi": 5059700,
            "previous_close_price": 153.6,
            "previous_oi": 4667700,
            "previous_volume": 1047989,
            "top_ask_price": 165,
            "top_ask_quantity": 375,
            "top_bid_price": 164.05,
            "top_bid_quantity": 50,
            "volume": 81097175
          }
        }
      }
    }
  }
```


Response Parameters:

Field	Type	Description
data.last_price	float	LTP of the Underlying
data.oc	array	Option Chain Array - Strike Wise
data.oc.{strike}	array	Strike Price for Underlying
data.oc.{strike}.ce	array	Call Option data of particular strike
data.oc.{strike}.pe	array	Put Option data of particular strike

Call/Put Option Data Parameters:

Field	Type	Description
greeks.delta	float	Measures the change of option\"s premium based on every 1 rupee change in underlying
greeks.theta	float	Measures how quickly an option\"s value decreases over time
greeks.gamma	float	Rate of change in an option\"s delta in relation to the price of the underlying asset

greeks.vega	float	Measures the change of option\"s premium in response to a 1% change in implied volatility
implied_volatility	float	Value of expected volatility of a stock over the life of the option
last_price	float	Last Traded Price of the Option Instrument
oi	int	Open Interest of the Option Instrument
previous_close_price	float	Previous day close price
previous_oi	int	Previous day Open Interest
previous_volume	int	Previous day volume
top_ask_price	float	Current best ask price available
top_ask_quantity	int	Quantity available at current best ask price
top_bid_price	float	Current best bid price available
top_bid_quantity	int	Quantity available at current best bid price
volume	int	Day volume for Option Instrument

Python Example:

```
dhan.option_chain(
  under_security_id=13,                # Nifty
  under_exchange_segment="IDX_I",
  expiry="2024-10-31"
)
```

Expiry List (`POST /optionchain/expirylist`)

Headers:
Header	Description
access-token required	Access Token generated via Dhan
client-id required	User specific identification generated by Dhan

Request Structure (JSON):

```
{
  "UnderlyingScrip":13,
  "UnderlyingSeg":"IDX_I"
}
```

Parameters:
Field	Field Type	Description
UnderlyingScrip required	int	Security ID of Underlying Instrument - can be found in

Instrument List |
| UnderlyingSeg | enum string | Exchange & segment of underlying for which data is to be fetched - Exchange Segment |

Response Structure:


```
{
  "data": [
    "2024-10-17",
    "2024-10-24",
    "2024-10-31",
    "2024-11-07",
    "2024-11-14",
    "2024-11-28",
    "2024-12-26",
    "2025-03-27",
    "2025-06-26",
    "2025-09-25",
    "2025-12-24",
    "2026-06-25",
    "2026-12-31",
    "2027-06-24",
    "2027-12-30",
    "2028-06-29",
    "2028-12-28",
    "2029-06-28"
  ],
  "status": "success"
}
```


Response Parameters:

Field	Type	Description
data[]	array	All expiry dates of underlying in `YYYY-MM-DD`

Python Example:

```
dhan.expiry_list(
  under_security_id=13,                # Nifty
  under_exchange_segment="IDX_I"
)
```

Instrument List API

You can fetch instrument list for all instruments which can be traded via Dhan by using the URLs below:

Compact: https://images.dhan.co/api-data/api-scrip-master.csv

Detailed: https://images.dhan.co/api-data/api-scrip-master-detailed.csv

These CSV files provide Security ID and other important details.

Segmentwise List

You can fetch detailed instrument list for all instruments in a particular exchange and segment by passing the same in parameters:

curl --location \'https://api.dhan.co/v2/instrument/{exchangeSegment}\' \

Python Example:

```
dhan.fetch_security_list("compact")
```

Column Description

Detailed tag	Compact tag	Description
EXCH_ID	SEM_EXM_EXCH_ID	Exchange NSE BSE MCX
SEGMENT	SEM_SEGMENT	Segment C - Currency D - Derivatives E - Equity M - Commodity
ISIN	-	International Securities Identification Number(ISIN) - 12-digit alphanumeric code unique for instruments
INSTRUMENT	SEM_INSTRUMENT_NAME	Instrument defined by

Detailed tag	Compact tag	Description
		Exchange - defined here
removed	SEM_EXPIRY_CODE	Expiry Code (applicable in case of Futures Contract) - defined Expiry Code
UNDERLYING_SECURITY_ID	-	Security ID of underlying instrument (applicable in case of derivative contracts)
UNDERLYING_SYMBOL	-	Symbol of underlying instrument (applicable in case of derivative contracts)
SYMBOL_NAME	SM_SYMBOL_NAME	Symbol name of instrument
removed	SEM_TRADING_SYMBOL	Exchange trading symbol of instrument
DISPLAY_NAME	SEM_CUSTOM_SYMBOL	Dhan display symbol name of instrument
INSTRUMENT_TYPE	SEM_EXCH_INSTRUMENT_TYPE	In addition to INSTRUMENT

Detailed tag	Compact tag	Description
		column, instrument type is defined by exchange adding more details about instrument
SERIES	SEM_SERIES	Exchange defined series for instrument
LOT_SIZE	SEM_LOT_UNITS	Lot Size in multiples of which instrument is traded
SM_EXPIRY_DATE	SEM_EXPIRY_DATE	Expiry date of instrument (applicable in case of derivative contracts)
STRIKE_PRICE	SEM_STRIKE_PRICE	Strike Price of Options Contract
OPTION_TYPE	SEM_OPTION_TYPE	Type of Options Contract `CE` - Call `PE` - Put
TICK_SIZE	SEM_TICK_SIZE	Minimum decimal point at which an instrument can be priced
EXPIRY_FLAG	SEM_EXPIRY_FLAG	

Detailed tag	Compact tag	Description
		Type of Expiry (applicable in case of option contracts) `M` - Monthly Expiry `W` - Weekly Expiry
BRACKET_FLAG	-	Bracket order status `N` - Not available `Y` - Allowed
COVER_FLAG	-	Cover order status `N` - Not available `Y` - Allowed
ASM_GSM_FLAG	-	Flag for instrument is ASM or GSM `N` - Not in ASM/GSM `R` - Removed from block `Y` - ASM/GSM
ASM_GSM_CATEGORY	-	Category of instrument in ASM or GSM `NA` in case of no surveillance
BUY_SELL_INDICATOR	-	Indicator to show if Buy and Sell is allowed in instrument `A`

Detailed tag	Compact tag	Description
		if both Buy/ Sell is allowed
BUY_CO_MIN_MARGIN_PER	-	Buy cover order minimum margin requirement (in percentage)
SELL_CO_MIN_MARGIN_PER	-	Sell cover order minimum margin requirement (in percentage)
BUY_CO_SL_RANGE_MAX_PERC	-	Buy cover order maximum range for stop loss leg (in percentage)
SELL_CO_SL_RANGE_MAX_PERC	-	Sell cover order maximum range for stop loss leg (in percentage)
BUY_CO_SL_RANGE_MIN_PERC	-	Buy cover order minimum range for stop loss leg (in percentage)

Detailed tag	Compact tag	Description
SELL_CO_SL_RANGE_MIN_PERC	-	Sell cover order minimum range for stop loss leg (in percentage)
BUY_BO_MIN_MARGIN_PER	-	Buy bracket order minimum margin requirement (in percentage)
SELL_BO_MIN_MARGIN_PER	-	Sell bracket order minimum margin requirement (in percentage)
BUY_BO_SL_RANGE_MAX_PERC	-	Buy bracket order maximum range for stop loss leg (in percentage)
SELL_BO_SL_RANGE_MAX_PERC	-	Sell bracket order maximum range for stop loss leg (in percentage)
BUY_BO_PROFIT_RANGE_MAX_PERC	-	Buy bracket order maximum

Detailed tag	Compact tag	Description
		range for target leg (in percentage)
SELL_BO_PROFIT_RANGE_MAX_PERC	-	Sell bracket order maximum range for target leg (in percentage)
BUY_BO_PROFIT_RANGE_MIN_PERC	-	Buy bracket order minimum range for target leg (in percentage)
SELL_BO_PROFIT_RANGE_MIN_PERC	-	Sell bracket order minimum range for target leg (in percentage)
MTF_LEVERAGE	-	MTF Leverage available (in x multiple) for eligible EQUITY instruments

Annexure

Exchange Segment

Attribute	Exchange	Segment	enum
IDX_I	Index	Index Value	0

Attribute	Exchange	Segment	enum
NSE_EQ	NSE	Equity Cash	1
NSE_FNO	NSE	Futures & Options	2
NSE_CURRENCY	NSE	Currency	3
BSE_EQ	BSE	Equity Cash	4
MCX_COMM	MCX	Commodity	5
BSE_CURRENCY	BSE	Currency	7
BSE_FNO	BSE	Futures & Options	8

Product Type

CO & BO product types will be valid only for Intraday.

Attribute	Detail
CNC	Cash & Carry for equity deliveries
INTRADAY	Intraday for Equity, Futures & Options
MARGIN	Carry Forward in Futures & Options
CO	Cover Order
BO	Bracket Order

Order Status

Attribute	Detail
TRANSIT	Did not reach the exchange server
PENDING	Awaiting execution
CLOSED	Used for Super Order, once both the entry and exit orders are placed
TRIGGERED	Used for Super Order, if Target or Stop Loss leg is triggered
REJECTED	Rejected by broker/exchange

Attribute	Detail
CANCELLED	Cancelled by user
PART_TRADED	Partial Quantity traded successfully
TRADED	Executed successfully

After Market Order time

Attribute	Detail
PRE_OPEN	AMO pumped at pre-market session
OPEN	AMO pumped at market open
OPEN_30	AMO pumped 30 minutes after market open
OPEN_60	AMO pumped 60 minutes after market open

Expiry Code

Attribute	Detail
0	Current Expiry/Near Expiry
1	Next Expiry
2	Far Expiry

Instrument

Attribute	Detail
INDEX	Index
FUTIDX	Futures of Index
OPTIDX	Options of Index
EQUITY	Equity
FUTSTK	Futures of Stock
OPTSTK	Options of Stock

Attribute	Detail
FUTCOM	Futures of Commodity
OPTFUT	Options of Commodity Futures
FUTCUR	Futures of Currency
OPTCUR	Options of Currency

Feed Request Code

Attribute	Detail
11	Connect Feed
12	Disconnect Feed
15	Subscribe - Ticker Packet
16	Unsubscribe - Ticker Packet
17	Subscribe - Quote Packet
18	Unsubscribe - Quote Packet
21	Subscribe - Full Packet
22	Unsubscribe - Full Packet
23	Subscribe - 20 Level Market Depth
24	Unsubscribe - 20 Level Market Depth

Feed Response Code

Attribute	Detail
1	Index Packet
2	Ticker Packet
4	Quote Packet
5	OI Packet

Attribute	Detail
6	Prev Close Packet
7	Market Status Packet
8	Full Packet
50	Feed Disconnect

Trading API Error

Type	Code	Message
Invalid Authentication	DH-901	Client ID or user generated access token is invalid or expired.
Invalid Access	DH-902	User has not subscribed to Data APIs or does not have access to Trading APIs. Kindly subscribe to Data APIs to be able to fetch Data.
User Account	DH-903	Errors related to User\"s Account. Check if the required segments are activated or other requirements are met.
Rate Limit	DH-904	Too many requests on server from single user breaching rate limits. Try throttling API calls.
Input Exception	DH-905	Missing required fields, bad values for parameters etc.
Order Error	DH-906	Incorrect request for order and cannot be processed.
Data Error	DH-907	System is unable to fetch data due to incorrect parameters or no data present.
Internal Server Error	DH-908	Server was not able to process API request. This will only occur rarely.
Network Error	DH-909	Network error where the API was unable to communicate with the backend system.
Others	DH-910	Error originating from other reasons.

Data API Error

Code	Description
800	Internal Server Error
804	Requested number of instruments exceeds limit
805	Too many requests or connections. Further requests may result in the user being blocked.
806	Data APIs not subscribed
807	Access token is expired
808	Authentication Failed - Client ID or Access Token invalid
809	Access token is invalid
810	Client ID is invalid
811	Invalid Expiry Date
812	Invalid Date Format
813	Invalid SecurityId
814	Invalid Request